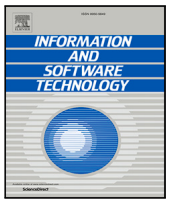
Information and Software Technology 184 (2025) 107758

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



Benchmarking large language models for automated labeling: The case of issue report classification

Giuseppe Colavito, Filippo Lanubile, Nicole Novielli *

University of Bari, Italy

ARTICLE INFO

Dataset link: https://github.com/collab-uniba/PromptingLLMs

Keywords:
Issue labeling
Generative AI
Software maintenance and evolution

ABSTRACT

Context: Issue labeling is a fundamental task for software development as it is critical for the effective management of software projects. This practice involves assigning a label to issues, such as *bug* or feature request, denoting a task relevant to the project management. To date, large language models (LLMs) have been proposed to automate this task, including both fine-tuned BERT-like models and zero-shot GPT-like models.
Objectives: In this paper, we investigate which LLMs offer the best trade-off between performance, response time, hardware requirements, and quality of the responses for issue report classification.
Methods: We design and execute a comprehensive benchmark study to assess 22 generative decoder-only LLMs and 2 baseline BERT-like encoder-only models, which we evaluate on two different datasets of GitHub issues.
Results: Generative LLMs demonstrate potential for zero-shot classification. However, their performance varies significantly across datasets and they require substantial computational resources for deployment. In contrast, BERT-like models show more consistent performance and lower resource requirements.
Conclusions: Based on the empirical evidence provided in this study, we discuss implications for researchers and practitioners. In particular, our results suggest that fine-tuning BERT-like encoder-only models enables achieving consistent, state-of-the-art performance across datasets even in presence of a small amount of labeled data available for training.

1. Introduction

Issue report classification is a largely investigated software engineering task, as it is critical for prioritizing and coordinating software development. This practice, adopted in software development, consists of assigning labels to issues denoting a category, such as bugs, feature requests, and other development tasks relevant to the project management. These labels help teams organize, prioritize, and track work effectively throughout the development lifecycle. Early work on issue classification focused on distinguishing between bugs and feature requests, using supervised machine learning and natural language processing (NLP) approaches for extracting text-based features [1,2]. More recently, encoder-only Large Language Models (LLMs), such as BERT [3] and its variants [4–6] have been used for classifying issue reports, achieving state-of-the-art performance [7–10]. However, the analysis of classification results revealed difficulties in accurately labeling issues belonging to minority classes [8–10], particularly when implementing fine-grained labeling schemes with limited data per class. The problem of data scarcity may be addressed by crowd-sourcing training data (e.g., from open-source projects) but crowd-sourced datasets may contain inconsistently labeled examples, thus introducing noise in the training [11].

To effectively address the data scarcity problem while ensuring the quality and internal consistency of labels in training data, we proposed using few-shot learning approaches with a reduced training set based on manually verified labels [7]. However, manual annotation remains time-consuming, even for small, curated datasets, underscoring the need to minimize these efforts while avoiding a performance drop. Furthermore, new projects have few, if any, issues that can be made available for annotation.

Leveraging billions of parameters, generative LLMs – i.e., models based on decoder-only transformers – may represent a viable solution to address the task of automated issue labeling in absence of training data. In fact, generative LLMs have already demonstrated their ability to generate code or assist developers for software engineering tasks they were not specifically trained for, and are now gaining increasing popularity also for tasks such as bug detection and code refactoring [12,13].

In our previous work [14], we investigated the use of gpt-3.5-turbo [15] for automated issue labeling in a zero-shot learning scenario, i.e. in the absence of training examples. We found that, without the

* Corresponding author.
E-mail address: nicole.novielli@uniba.it (N. Novielli).

https://doi.org/10.1016/j.infsof.2025.107758
Received 20 August 2024; Received in revised form 9 April 2025; Accepted 15 April 2025
Available online 25 April 2025

need for fine-tuning, GPT-like LLMs can achieve a performance comparable to BERT-like LLMs. As a consequence, we concluded that, whenever a gold standard dataset is not available, the issue classification task might still be addressed successfully in a zero-shot setting, achieving results comparable to the state of the art. However, these findings cannot be generalized beyond a single model and the single dataset used for our former study. Furthermore, being proprietary, GPT-like models may introduce unwanted external dependencies and trigger privacy-related concerns in case of issues including sensitive data. In such cases, on-premise generative LLMs might be required, thus posing critical deployment challenges due to computational and scalability issues. A cost analysis, in terms of computational resources, should always be conducted to assess whether deploying generative LLMs is a viable solution in practice.

This study aims to fill the aforementioned gaps in the literature. Specifically, we build upon and complement the findings of our previous work on the use of LLMs for automated issue labeling using BERT-like [16] and generative LLMs [14]. As such, we formulate our first research question as follows:

RQ: Which LLMs offer the best trade-off between performance, response time, hardware requirements, and quality of the responses for issue report classification?

To address our RQ, we perform a comprehensive evaluation of 22 generative Large Language Models (LLMs) for automated issue labeling in software development, comparing them against encoder-only BERT-like models. By including several LLMs in multiple sizes, evaluated on two different datasets of GitHub issues, we assess how consistent the behavior of zero- and few- shot classification is across different models and datasets. Such an extended comparison provides valuable insights to researchers and practitioners, especially in light of the trade-offs posed by the computational costs associated with the deployment of on-premise LLMs and labeling costs for the training of BERT-like models.

As such, beyond assessing the classification performance, we also provide an assessment of response time for LLMs of different sizes, as using LLMs for building classifiers might pose critical challenges at serving time. To this aim, we complement the performance assessment in terms of precision, recall, and f-measure with a cost analysis in terms of computational resources.

In this study, we leverage two distinct datasets of GitHub issues for evaluation: the first includes a manually verified dataset with strong inter-rater agreement, while the second is a larger dataset containing 1500 issues with original labels as collected from the corresponding five major GitHub projects they belong to. This dual-dataset approach enables a thorough assessment of how consistently models perform across-dataset that might differ for internal consistency of labels.

The main contributions of this study can be summarized as follows:

- We enhance the current understanding of how LLMs can be leveraged to address the task of automatic issue report classification. Specifically, we complement and extend our previous work on the assessment of GPT-like models for issue labeling by extending our empirical evaluation to include generative models requiring on-premise deployment. Beyond evaluating the classification performance through precision, recall, and F1 scores, we provide an assessment of the inference time, computational requirements, and the models' ability to follow instructions. We provide detailed information about hardware requirements, comparing configurations using different numbers of GPUs, and explore variations in the inference time and the challenges associated with parsing model outputs. By examining these multiple dimensions simultaneously, our study provides a broad picture of the current state of automated issue labeling, thus enhancing the current understanding of both the need for achieving a good classification performance and the practical implications of choosing different LLMs for automated issue labeling;
- We show that generative AI might not always represent a viable solution to address the task of automated issue labeling in the absence of training data, due to inconsistent performance across datasets. Conversely, we provide evidence that the fine-tuning of BERT-like models, using a minimal amount of training data, achieves an optimal classification performance without requiring high computational resources;
- Based on the empirical results reported in this paper, we offer a discussion on how to address automated issue labeling in a low-resource setting, discussing the trade-off between the need to reduce the human costs associated with the manual labeling of training data for fine-tuning and the computational costs associated with the deployment of LLMs with billions parameters;
- As a final contribution, we built and released a fine-tuned BERT-like sentence transformer model specifically designed for analyzing GitHub issues.[1] Furthermore, to encourage future empirical studies, we built and distributed a replication package, which is publicly available for reuse.[2]

The remainder of this paper is organized as follows. In Section 2, we provide background information on LLMs and an overview of their use in software engineering research, specifically focusing on automated issue labeling. In Section 3, we present the datasets and the methodology we adopt do address our research question. We outline the study results in Section 4 and discuss our findings in Section 5. Finally, we discuss the study limitations in Section 6 and conclude in Section 7.

2. Background and related work

In this section, we provide background knowledge on LLMs, report an overview of related work on their usage in software engineering, and present the state of the art on automated issue labeling. Finally, we conclude by highlighting the novel contribution of this study.

2.1. Large language models

The advent of accessible GPU hardware has enabled transformer-based LLMs [17] to revolutionize NLP, establishing new state-of-the-art benchmarks across numerous tasks [3,18,19]. These models leverage self-attention mechanisms [17] to process input sequences by dynamically focusing on relevant contextual information when representing each token, surpassing the limitations of earlier architectures like RNNs [20] and LSTMs [21] in capturing long-range dependencies. The effectiveness of LLMs stems primarily from their pre-training approach, which enables them to learn linguistic patterns from vast collections of unlabeled text [3,22].

The transformer architecture has given rise to numerous LLMs, with BERT [3] emerging as a pivotal encoder-only model. The architecture of BERT incorporates a dual training approach, which combines masked language modeling, where the model predicts contextually masked tokens, with next-sentence prediction to determine sequential relationships between sentences. After BERT was released, many variants have been proposed [4–6,23]. BERT-like models can be further fine-tuned on downstream tasks, achieving state-of-the-art performance in many NLP challenges [3,6].

The success of BERT-like models has fostered research in this field, thus leading to the development of larger models. It is the case of decoder-only models, also known as generative models, which include billions of parameters. Generative LLMs, – e.g., the GPT family [15, 18,24], and open-source decoder-only models such as LLaMA [25, 26], PaLM [27], Gemini [28], Vicuna [29], Falcon [30], Mistral [31],

[1] The model is publicly available on Hugging Face at https://huggingface.co/Collab-uniba/github-issues-mpnet-st-e10

[2] https://github.com/collab-uniba/PromptingLLMs

Table 1
Distribution of manually validated labels for the issues extracted from the NLBSE'23 dataset.

Manually validated dataset				
Label	Train set		Test set	
Bug	47	24%	53	27%
Documentation	33	17%	32	16%
Feature	60	30%	55	28%
Question	44	22%	47	24%
Total	184		187	

Zephyr [32] – have demonstrated the power and versatility of the transformer architecture when scaling up the number of parameters [33]. In particular, they exhibit emergent abilities that arise suddenly at large scales and cannot be extrapolated from smaller models. These include few-shot learning on diverse NLP tasks, multi-step reasoning, question answering using world knowledge, and instruction following. The mechanisms behind emergence are not fully understood, but hypothesized factors include model capacity, depth, and ability to leverage huge amounts of pre-training data [34].

2.2. LLMs in software engineering

Traditional software engineering tasks involving natural language analysis have been addressed using encoder-only LLMs, with BERT [3] and its variants [4–6] being prominent examples. To better address software-specific needs, specialized versions such as CodeBERT [35] and BERTOverflow [36] have been developed. Expanding beyond encoder-only architectures, researchers have also investigated the application of encoder–decoder LLMs in SE tasks. Notable examples in this category include T5 [37] and its code-oriented variant, CodeT5 [38].

Since the release of GPT 3 [24] and GPT 3.5 [15], there has been a notable shift towards decoder-only models to leverage the potential of LLMs in addressing software engineering challenges [12,13]. These models have demonstrated impressive abilities in generating code and providing code completions [39,40]. According to GitHub, 46% of the developers' code was written by Copilot, improving the pace of writing code [12,41]. Such generative models have also been used in a wide range of tasks, such as bug localization and repair [42–46], code refactoring [47,48], code translation [49,50], code summarization [51,52], software testing [53], code clone detection [54], and code comprehension [55].

2.3. Issue report classification

Issue tracking is a widely adopted practice to manage the evolution of software products. Issue tracking systems allow contributors to report bugs, request new features, ask questions about the software product, but also find some tasks to work on. However, especially for open source projects, the use of such tools complicates the work of developers and maintainers, as they have to deal with a large number of issues submitted every day, with different types and quality [56–58]. Issue reports typically contain a title and a description, the issue status (e.g., open, assigned, closed), a comment thread, and a label indicating the type of issue. Through the issue tracking system, maintainers can assign issues to the most appropriate team member, prioritize them, and track their resolution.

Issue labels serve as a powerful classification mechanism that enables effective project management and prioritization [59,60]. Through both predefined and custom labels, maintainers can quickly identify an issue's topic, associated development tasks, and priority level, streamlining the workflow through efficient filtering and organization.

Previous research [1,61] shows how developers and maintainers often assign an incorrect issue label to the reports. Furthermore, the labeling mechanism is rarely used by contributors at the time the issue is created [56,62], thus requiring additional effort for manually assigning labels to issue reports [57]. To reduce this effort, several approaches to the automatic categorization of issue reports have been proposed using supervised machine learning [1,61–65].

More recently, empirical studies demonstrated that BERT-like models can be effectively employed for the issue report classification problem [8,10,66,67], thus currently representing the state of the art. With the advent and growing popularity of generative AI, researchers have started investigating to what extent decoder-only LLMs, such as GPT-like models, can be leveraged to automate issue labeling [14,68].

2.4. Novel contributions

In this paper, we complement and extend the body of knowledge on the use of LLMs for software engineering tasks by investigating to what extent we can successfully automate issue labeling using LLMs. This paper builds upon and extends previous work by providing a comprehensive evaluation of the performance of generative LLMs to address issue labeling tasks in a low-resource setting, i.e., when a high-quality, manually annotated gold standard dataset is not available for fine-tuning. Our study encompasses diverse model families, including open-source models (like LLaMA or Mistral), code-specific models, such as CodeLLaMA, and proprietary models that can be used through API calls (GPT-4), thus providing a comprehensive evaluation of generative models for issue labeling. Specifically, we compare the classification performance of 22 generative LLMs for automated issue labeling in software development, comparing them with BERT-like encoder-only models. We further complement our comparison by offering an investigation of the trade-off between the need for human costs associated with labeling of training data required for fine-tuning BERT-like models and the computational costs associated with the usage of generative LLMs, which can also be used in a zero-shot setting (i.e., in the absence of training data). The study uses two distinct datasets of GitHub issues for evaluation: a manually verified dataset and a larger dataset including original labels collected from their source GitHub projects. This dual-dataset approach allows assessment of model performance consistency across different data contexts.

3. Methodology

The present benchmark study measures the performance of BERT-like and generative LLMs, including both open-source and GPT-like proprietary models, on issues collected from GitHub. In the following, we describe the datasets we used, the models included in the benchmark, and the prompting strategy to interact with the models.

3.1. Datasets

We use two datasets of GitHub issues from open source software projects, which had been collected to support the issue-classification challenges of two editions of the Natural Language-Based Software Engineering (NLBSE) workshop series. As a first dataset, we select the same dataset used in our previous study to evaluate the performance of GPT-like models [7], to ensure that the results of the present study are comparable. This dataset was created in the scope of our previous research and is publicly available [7]. It is created by sampling 400 issues from the NLBSE'23 dataset [16], and then manually labeling them. While limited in size, this dataset represents a high-quality gold standard, as the labels were verified by human annotators with a Cohen $\kappa = 0.74$, which corresponds to a substantial inter-rater agreement [69]. The dataset contains issues with the following labels: *bug*, *documentation*, *feature*, and *question*. In Table 1, we report the distribution of the labels in the train and test sets after the hand labeling of two initial train and test sets containing 200 issues each. As a results of the disagreement resolution, some issues were discarded from the labeled dataset of 400 issues. Specifically, 16 (8%) issues were removed

Table 2
Distribution of labels for the NLBSE'24 Dataset. The labels are the same collected from the source repositories.

NLBSE'24 dataset				
Label	Train set		Test set	
Bug	500	33%	500	33%
Feature	500	33%	500	33%
Question	500	33%	500	33%
Total	1500		1500	

from the train set and 13 (7%) from the test set, as the raters could not agree on the label to assign to these issues.

As a second dataset, we use the GitHub issues dataset from the NLBSE'24 competition [70]. The projects cover a wide range of domains, including web development, machine learning, software development tools, cryptocurrency, and computer vision. The dataset contains issues with the following labels: *bug*, *feature*, and *question*. Specifically, this dataset contains issues from 5 large GitHub projects, namely *facebook/react*, *tensorflow/tensorflow*, *microsoft/vscode*, *bitcoin/bitcoin*, and *OpenCV/opencv*. The dataset is already divided into training and test sets by its original authors, with a 50–50 stratified split. Each label has the same number of samples from each project (100). In Table 2, we report the distribution of the labels for this dataset.

For both datasets, we use the train set portion only for training the BERT-like baseline models (see Section 3.6) and for selecting the example to be included in the prompt for the evaluation of generative LLMs in the few-shot setting (see Section 3.3).

The choice of the datasets to include in this study is in line with our research goals. First, by including the consideration of the first dataset, we enable a direct comparison of the performance of all models included in the current study with the GPT 3.5 performance we observed in our previous study [14]. Furthermore, although the size of the first dataset is limited, it is entirely validated through manual annotation with high observed inter-rater agreement [16]. This choice is in line with the findings of previous research on the impact of data quality in automated issue labeling, suggesting that limited classification performance could be due to a poor internal consistency of labels [11]. On the other hand, the second dataset is almost four times the size of the first one and includes the original labels reported in the source GitHub projects considered for data collection, thus allowing to generalize results with respect to the dataset dimension and labeling criteria. In particular, the choice of including a second dataset with raw, unverified labels, allows us to evaluate the classifiers in a setting in which there are no resources for building a manually validated dataset.

3.2. Models

In our study, we consider different models including open-source LLMs (like LLaMA or Mistral), code-specific models (such as CodeLLaMA), and proprietary models that can be used through API calls (GPT-4), thus providing a comprehensive evaluation of generative models for issue-labeling.

As for generative models that require on-premise deployment, we consider 22 of different sizes, ranging from $\sim$ 7 billion parameters to $\sim$ 70 billion parameters (see Table 3). We select models that are freely available to the research community through the Hugging Face model hub [71]. Specifically, we include not only LLMs for instruction following, but also LLMs for code generation because the issues often contain code snippets and, in general, code-related text. We choose models based on their popularity and availability on the Hugging Face model hub. Specifically, we include models that are provided by well-known organizations like Google, Meta, MistralAI, BigCode, Alibaba Cloud, and DeepSeek.

We divide the models into two groups based on the hardware used for the benchmark. The first group includes models that fit into 2 Nvidia A100 64 GB GPUs, while the second group includes models that fit into 4 Nvidia A100 64 GB GPUs. All the aforementioned models have been deployed on premise. To reduce the memory footprint and speed up the inference time, all models are 4-bit quantized before being prompted. The quantization process is performed using the bitsandbytes library [72].

For completeness, we include models that can be accessed through the OpenAI API. Specifically, we consider the latest GPT model at the time of writing, i.e., GPT-4o [18]. For the sake of comparison with our previous work, we also report the performance of gpt-3.5-turbo on the manually verified dataset [14]. Gpt-3.5-turbo could not be assessed on the second dataset included in this benchmark as it is not available anymore at the time of the study.

3.3. Prompting

The prompt template is structured to provide the model with the necessary information to classify the issue report effectively, including the task description, label descriptions, and the input issue to be classified. We also ask the model to provide step-by-step reasoning before assigning the label, trying to make the model build a chain-of-thought, a technique that has been shown to improve the performance of generative models [73].

Our prompt template, illustrated in Fig. 1, consists of the following components:

- **Input Format:** Specifies the structure of the issue report, including title and body;
- **Task Description:** Outlines the classification task and available labels;
- **Label Descriptions:** Provides definitions for each class label, derived through a semi-automatic process involving ChatGPT and manual verification [14];
- **Examples:** Provides examples of issues for each label. This part is used only for few-shot learning, where the model is prompted with labeled examples.
- **Input Issue:** The specific issue to be classified;
- **Output Format Instructions:** Specifies the desired format for the model predictions.

We adopt both zero-shot and few-shot learning strategies. In the zero-shot setting, the prompt only contains the label descriptions, without including any labeled examples. In the few-shot setting, we experiment with 1- and 2-shot settings, including one and two examples per class, respectively. In line with previous work in this field using LLMs for classification tasks in software engineering [14,74], we randomly select the examples to include in the prompt from the train set split for each dataset.

3.4. Parsing the output of generative models

We ask the models to produce a JSON object containing the label and the reasoning for the assignment. The models do not always format the output correctly. Some models tend to forget commas or closing brackets, which makes it difficult to parse the output. In some cases, the output is a set of two or more JSON objects. In other cases, the output is a JSON followed by a string. This is why we use an extraction strategy based on regular expressions and provide the percentage of outputs that are correctly formatted as requested in the prompt. We search for the first occurrence of the pattern `"label": "".*"` and extract the label from the match. If the pattern is not found, we consider the output as not valid.

We provide the percentage of outputs correctly formatted as requested in the prompt, in order to give evidence of the quality of the responses of the models. Similar issues with inconsistent output formats in LLMs have been explored in a recent study on code translation tasks [49], which highlights the importance of addressing these inconsistencies with specific extraction methods for accurate model evaluation and benchmarking.

Table 3
Generative models included in the benchmark.

	Hardware used			
	2xA100 64 GB		4xA100 64 GB	
Organization	Model name	Params	Model name	Params
bigcode	starcoder2-7b starcoder2-15b	7b 15b	–	
codellama	CodeLlama-7b-Instruct-hf CodeLlama-13b-Instruct-hf	7b 13b	CodeLlama-34b-Instruct-hf CodeLlama-70b-Instruct-hf	34b 70b
deepseek-ai	deepseek-coder-6.7b-instruct deepseek-coder-7b-instruct-v1.5	6.7b 7b	deepseek-coder-33b-instruct	33b
google	gemma-7b-it	7b	–	
HuggingFaceH4	zephyr-7b-beta	7b	–	
meta-llama	Llama-2-7b-chat-hf Llama-2-13b-chat-hf Meta-Llama-3-8B-Instruct	7b 13b 8b	Llama-2-70b-chat-hf Meta-Llama-3-70B-Instruct	70b 70b
mistralai	Mistral-7B-Instruct-v0.2 Mistral-7B-Instruct-v0.3	7b 7b	Codestral-22B-v0.1 Mixtral-8 × 7B-Instruct-v0.1	22b 46b
Qwen	Qwen2-7B-Instruct	7b	Qwen2-72B-Instruct	72b

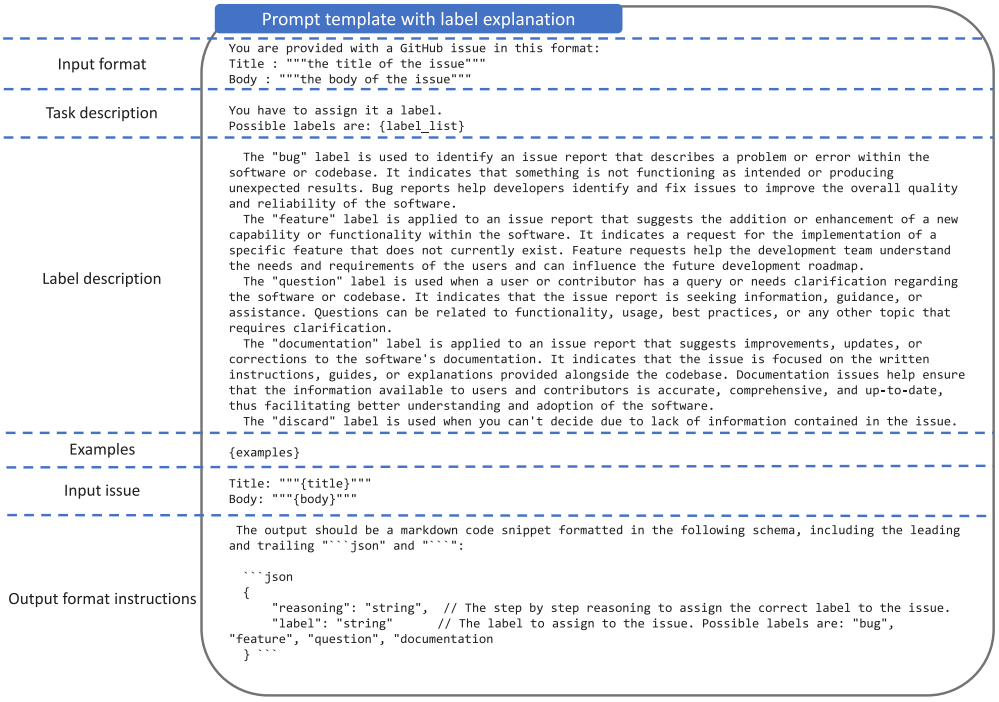


Fig. 1. Structure of the prompt template used for this study. The picture is replicated from our previous work [14], from which we replicate the prompting strategy.

3.5. Evaluation

To evaluate the performance of the models, we use the F1 score, which is the harmonic mean of precision and recall. We provide the F1 score for each label, as well as the overall micro- and macro-averaged F1 scores. F1-micro is calculated by first computing the global contingency table by summing over all label-specific contingency tables, and then calculating F1 using these global sums. In other words, it aggregates the classification outputs first before calculating F1. F1-macro is calculated by first computing F1 for each individual label separately, and then taking the average of these label-specific F1 scores. While micro-averaging gives equal weight to each issue-label decision pair, macro-averaging gives equal weight to each category. Whether F1-micro or F1-macro should be used depends on the envisioned use-case scenario: micro-averaging is known to be influenced by the performance on the majority class. On the other hand, the ability of a classifier to correctly classify items belonging to classes with fewer instances is better represented by macroaveraging [75].

To provide a more complete and accurate evaluation of the classification performance, we provide both metrics, and we rank the models focusing on the F1-macro score, as it provides a balanced evaluation of the performance of the models across all labels, even when the dataset is imbalanced. Finally, we provide the models' response time. Specifically, we report the time that each model takes to classify the entire test set.

3.6. Baselines

We compare the performance of the generative models with two BERT-like baselines: RoBERTa [6] and SETFIT [76]. To determine the significance of the difference between the classification performance of the generative models and the baselines, we applied the McNemar, a statistical test used to determine if there are differences between paired nominal data. In machine learning, it is broadly used for comparing the performance of two classification models on the same test set [77].

Table 4
Performance of LLMs on NLBSE'23 manually verified test set in the zero-shot setting. The on-premise models are grouped by the hardware required. The best classification performance, response time and percentage of parsable output are highlighted in bold.

		2xA100 64 GB							
		F1							
Model	**Params**	**Bug**	**Docs**	**Feat**	**Ques**	**Micro**	**Macro**	**Time**	**% Parsable**
Meta-Llama-3-8B-Instruct	8b	**0.85**	0.73	**0.85**	**0.74**	**0.80**	**0.79**	01:11:04	67
Mistral-7B-Instruct-v0.3	7b	0.81	0.77	0.83	0.64	0.78	0.76	00:11:55	1
Qwen2-7B-Instruct	7b	0.78	**0.79**	0.80	0.59	0.75	0.74	**00:09:03**	**98**
Mistral-7B-Instruct-v0.2	7b	0.81	0.68	0.71	0.68	0.73	0.72	00:13:10	0
Llama-2-13b-chat-hf	13b	0.82	0.62	0.82	0.59	0.74	0.71	00:12:34	87
deepseek-coder-7b-instruct-v1.5	7b	0.79	0.67	0.78	0.56	0.72	0.70	00:17:40	92
CodeLlama-7b-Instruct-hf	7b	0.71	0.57	0.70	0.58	0.65	0.64	00:49:42	3
gemma-7b-it	7b	0.68	0.68	0.68	0.51	0.65	0.64	00:16:14	76
deepseek-coder-6.7b-instruct	6.7b	0.71	0.48	0.74	0.45	0.64	0.59	01:10:49	49
zephyr-7b-beta	7b	0.73	0.63	0.56	0.41	0.62	0.58	00:38:59	0
CodeLlama-13b-Instruct-hf	13b	0.57	0.75	0.58	0.38	0.57	0.57	01:27:23	44
Llama-2-7b-chat-hf	7b	0.68	0.53	0.59	0.42	0.59	0.56	00:12:51	79
starcoder2-15b	15b	0.50	0.52	0.62	0.51	0.54	0.54	01:14:18	0
starcoder2-7b	7b	0.59	0.43	0.43	0.27	0.45	0.43	00:59:00	0
		4xA100 64 GB							
Qwen2-72B-Instruct	72b	**0.85**	**0.92**	**0.90**	**0.83**	**0.87**	**0.88**	02:56:31	0
Meta-Llama-3-70B-Instruct	70b	0.84	0.85	0.85	0.80	0.84	0.84	00:26:26	**100**
Codestral-22B-v0.1	22b	0.79	0.85	0.81	0.73	0.80	0.80	**00:23:13**	88
Mixtral-8 × 7B-Instruct-v0.1	46b	0.82	0.73	0.76	0.74	0.77	0.76	00:36:18	96
Llama-2-70b-chat-hf	70b	0.81	0.69	0.82	0.64	0.76	0.74	00:51:20	98
CodeLlama-70b-Instruct-hf	70b	0.80	0.63	0.77	0.46	0.70	0.67	03:05:20	2
CodeLlama-34b-Instruct-hf	34b	0.73	0.57	0.56	0.59	0.63	0.61	00:28:18	37
deepseek-coder-33b-instruct	33b	0.69	0.46	0.64	0.18	0.56	0.49	00:30:39	8
		API Call							
gpt-4o-2024-05-13	–	0.80	0.86	0.86	0.73	0.81	0.81	00:07:34	100
gpt-3.5-turbo-16k-0613	–	0.83	0.73	0.87	0.82	0.82	0.81	00:05:00	100

In this study, we perform the McNemar test with $\alpha = 0.05$ and Bonferroni correction, to account for repeated pairwise comparisons on the same dataset. In the following, we comment only on statistically relevant differences. For the sake of space, the detailed matrices with all p-values are included in the replication package.

Both RoBERTa- and SETFIT-based classifiers were assessed in our previous work [7] and were selected as baselines in the NLBSE'24 tool competition [70]. They are both built using the training set of each dataset included in this benchmark (see Section 3.1). We used the default learning rate of 7×10^{-6}, a batch size of 16, and trained for 20 epochs, consistent with the settings in [78] for the RoBERTa-base model.

As regards SETFIT, we use two sentence embedding models, namely *sentence-transformers/all-mpnet-base-v2* and *Collab-uniba/github-issues-mpnet-st-e10* as base sentence embedding models, both with a batch size of 16, a single epoch of training and a number of iterations of 20 as specified in [7].

The first model is selected as it is one of the most widely used among the publicly available SETFIT pre-trained models. These sentence-transformers models, including *sentence-transformers/all-mpnet-base-v2*, are trained on general text. However, technical texts can be quite different from general-purpose ones, as they may contains code snippets or domain-specific lexicon. Furthermore, the pool of generative models evaluated in this study (see Section 3.2) also includes models such as *bigcode* or *codellama* that are trained with data from the software development domain.

To enable a fair comparison of the BERT-like baselines with the generative models included in this benchmark, we also include the performance of the SETFIT classifier using the *Collab-uniba/github-issues-mpnet-st-e10*, an additional sentence-transformer model that we built by fine-tuning the using microsoft/mpnet-base with the dataset of the NLBSE'22 issue-labeling challenge [79]. To build it, we used the train set created by the challenge organizers, containing 722,899 issues describing *bugs*, requests for *enhancement*, and *questions*. We removed duplicates and issues with an empty body, thus obtaining a set of 627,521 issues that we used for fine-tuning the *Collab-uniba/github-issues-mpnet-st-e10* sentence-transformer model. Our model is obtained using the *microsoft/mpnet-base* [80] as the base model, and by continuing the training procedure using both the title and body of the issue reports. The model is trained with a batch size of 16 and a learning rate of 2e−05, for a total of 10 training epochs. The resulting sentence-transformers model is publicly available for reuse on Hugging Face.[3]

4. Results

In this section, we present the results obtained in the zero- and few-shot learning on the two datasets we consider for this study.

4.1. Performance with zero-shot learning

We first provide an overview of the performance of the models on the manually verified dataset (see Table 4), and then we present the results on the NLBSE'24 dataset (see Table 6). We also provide the response time for each model and an analysis of the quality of the responses, focusing on the model's ability to correctly follow instructions. The generative models deployed on premise are grouped in two categories based on the minimum hardware required, i.e., 2xA100 64 GB vs.4xA100 64 GB. All the models that could not fit in the 2xA100 64 GB hardware are included in the 4xA100 64 GB group. For each group, the models are sorted based on the macro F1 score.

Manually labeled test set from NLBSE '23. In Table 4, we report the performance obtained by the models on the manually verified dataset. For the first group of models (2xA100 64 GB), the best performing model is Meta-Llama-3-8B-Instruct (macro F1 score = .79).

[3] https://huggingface.co/Collab-uniba/github-issues-mpnet-st-e10

Table 5
Comparison of the performance of BERT-like baselines (SETFIT and RoBERTa) and the top generative LLMs achieving the best classification performance on the NLBSE'23 test set. The best F1 values by class and overall are highlighted in bold.

	BERT-like models									Generative models (zero-shot)						
	RoBERTa			SETFIT (base)			SETFIT (issues)			Meta-Llama-3-8B			Qwen2-72B			Label
Label	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**% Distrib.**
Bug	0.88	0.73	0.80	0.87	0.85	0.86	0.90	0.85	**0.87**	0.75	0.98	0.85	0.75	0.98	0.85	28
Documentation	0.64	0.50	0.56	0.90	0.66	0.76	0.92	0.75	**0.83**	0.88	0.64	0.74	0.95	0.74	**0.83**	17
Feature	0.68	0.76	0.72	0.75	0.92	0.83	0.79	0.95	0.86	0.90	0.80	0.85	0.94	0.85	**0.90**	30
Question	0.75	0.89	0.82	0.88	0.83	0.85	0.93	0.89	0.91	0.71	0.75	0.73	0.94	0.91	**0.92**	25
Micro avg	0.74	0.74	0.74	0.83	0.83	0.83	0.87	0.87	**0.87**	0.81	0.80	0.80	0.87	0.87	**0.87**	
Macro avg	0.75	0.74	0.74	0.85	0.81	0.82	0.88	0.87	0.87	0.81	0.79	0.79	0.89	0.87	**0.88**	
Hardware	1xA100 64 GB									2xA100 64 GB			4xA100 64 GB			
Training Time	0:02:03			0:03:35			0:04:53			–			–			
Inference Time	0:00:01			0:00:01			0:00:01			01:11:04			2:56:31			

Table 6
Performance of LLMs on the NLBSE'24 test set in the zero-shot setting. The on-premise models are grouped by the hardware required. The best classification performance, response time, and percentage of parsable output are highlighted in bold.

		2xA100 64 GB						
		F1						
Model	**Params**	**Bug**	**Feat**	**Que**	**Micro**	**Macro**	**Time**	**% Parsable**
Meta-Llama-3-8B-Instruct	8b	**0.70**	0.80	**0.48**	**0.68**	**0.66**	09:01:25	73
Mistral-7B-Instruct-v0.2	7b	0.70	0.78	0.43	0.66	0.64	01:48:04	<1
Mistral-7B-Instruct-v0.3	7b	0.70	**0.81**	0.40	0.66	0.63	01:41:38	<1
deepseek-coder-7b-instruct-v1.5	7b	0.68	0.81	0.39	0.65	0.63	02:33:28	86
Qwen2-7B-Instruct	7b	0.69	0.79	0.39	0.65	0.62	**01:14:54**	**99**
CodeLlama-7b-Instruct-hf	7b	0.69	0.77	0.40	0.65	0.62	05:19:31	2
Llama-2-13b-chat-hf	13b	0.67	0.78	0.34	0.64	0.59	01:48:25	75
deepseek-coder-6.7b-instruct	6.7b	0.64	0.71	0.26	0.58	0.54	09:36:07	42
CodeLlama-13b-Instruct-hf	13b	0.62	0.60	0.34	0.55	0.52	11:12:45	29
zephyr-7b-beta	7b	0.62	0.63	0.26	0.55	0.50	05:31:47	0
starcoder2-7b	7b	0.52	0.58	0.39	0.50	0.50	08:21:14	<1
gemma-7b-it	7b	0.60	0.65	0.17	0.53	0.47	02:26:55	80
Llama-2-7b-chat-hf	7b	0.57	0.57	0.22	0.50	0.47	02:14:10	52
starcoder2-15b	15b	0.53	0.56	0.20	0.46	0.43	10:17:12	<1
		4xA100 64 GB						
Mixtral-8 × 7B-Instruct-v0.1	46b	**0.73**	0.79	**0.52**	**0.69**	**0.68**	04:58:03	97
Meta-Llama-3-70B-Instruct	70b	0.72	0.81	0.50	0.69	0.68	04:00:49	92
Qwen2-72B-Instruct	72b	0.71	**0.82**	0.47	0.69	0.67	03:59:18	**99**
Codestral-22B-v0.1	22b	0.69	0.81	0.41	0.66	0.64	**03:19:12**	71
CodeLlama-70b-Instruct-hf	70b	0.67	0.78	0.44	0.65	0.63	25:17:10	2
Llama-2-70b-chat-hf	70b	0.67	0.76	0.42	0.63	0.61	07:09:29	97
CodeLlama-34b-Instruct-hf	34b	0.65	0.57	0.50	0.58	0.58	03:31:19	18
deepseek-coder-33b-instruct	33b	0.65	0.74	0.26	0.60	0.55	04:44:03	9
		API Call						
gpt-4o-2024-05-13	–	0.69	0.80	0.41	0.66	0.63	**01:00:41**	100

Despite having the best classification performance, the model has a substantially higher response time (1 h 11' 4") with respect to the majority of the models in the same category. The model is able to correctly follow the output formatting instructions in 67% of the cases. Sacrificing .05 of macro F1 score, it is possible to obtain the best response time (9' 3") and amount of parsable responses (98%) of the group using the Qwen2-7B-Instruct model, which achieves comparable performance (F1 micro = .74), albeit with a significant drop in the F1 for the *question* class (F1 = .59, compared to .74 obtained by Meta-Llama-3-8B-Instruct).

For the second group of models (4xA100 64 GB), the model achieving the best classification performance is Qwen2-72B-Instruct (F1 macro = .88). However, the model has an higher response time (2 h 56' 31") with respect to the majority of the models in the same category and it is not able to correctly follow the instructions in the prompt, thus requiring post-processing of the output. Instead, the Meta-Llama-3-70B-Instruct model has a competitive response time (26' 26") while achieving a comparable classification performance (F1 macro = .84). Furthermore, it can produce 100% of parsable responses.

Meta-LLama-3-8B-Instruct achieves comparable performance to gpt-4o-2024-05-13 in terms of macro F1 score (.79 vs. .81), while Qwen2-72B-Instruct outperforms it (.88). It is important to note that GPT models are always capable of correctly following the instructions on how to format the output and require lower response time.

In Table 5, we report the generative models achieving the best classification performance for each group and compare them with the RoBERTa (F1 macro = .74) and SETFIT baselines (F1 macro = .82 for the classifier using the base sentence-transformer model pre-trained on general texts and F1 macro = .87 for the classifier using the sentence-transformer model fine-tuned on issues). For the 2xA100 64 GB group, Meta-LLama-3-8B-Instruct achieves a better classification performance (F1 macro = .79) compared to the RoBERTa baseline in terms of macro F1 score, albeit the difference in F1 is not statistically significant. Conversely, it does not outperform any of the SETFIT ones. It is worth noting that the SETFIT baseline requires supervised training, which

Table 7
Comparison of the performance of BERT-like baselines (SETFIT and RoBERTa) and the top-performing generative LLMs on the NLBSE'24 test set.

	BERT-like models									Generative models (zero-shot)						
	RoBERTa			SETFIT (base)			SETFIT (issues)			Meta-Llama-3-8B			Mixtral-8 × 7B			Label
Label	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**% Distrib.**
Bug	0.80	0.78	0.79	0.82	0.82	0.82	0.83	0.85	**0.84**	0.57	0.89	0.70	0.60	0.91	0.73	33
Feature	0.80	0.85	0.82	0.83	0.88	**0.85**	0.83	0.85	0.84	0.83	0.77	0.80	0.85	0.74	0.79	33
Question	0.76	0.74	0.75	0.79	0.75	0.77	0.81	0.77	**0.79**	0.72	0.36	0.48	0.70	0.41	0.52	33
Micro avg	0.79	0.79	0.79	0.82	0.82	**0.82**	0.82	0.82	**0.82**	0.68	0.67	0.68	0.69	0.69	0.69	
Macro avg	0.79	0.79	0.79	0.82	0.82	**0.82**	0.82	0.82	**0.82**	0.71	0.67	0.68	0.72	0.69	0.68	

Hardware	1xA100 64 GB			2xA100 64 GB		4xA100 64 GB	
Training Time	0:12:15	0:29:30	0:40:47	–		–	
Inference Time	0:00:09	0:00:08	0:00:09	09:01:25		04:58:03	

Table 8
Confusion matrix for the two best-performing generative LLMs on the manually validated labels for the issues extracted from the NLBSE'23 dataset. Rows represent actual classes, while columns correspond to predicted labels.

Meta-Llama-3-8B-Instruct	Bug	Feature	Documentation	Question	*Qwen2-72B-Instruct*	Bug	Feature	Documentation	Question
Bug	98%	–	2%	–	Bug	98%	–	2%	–
Feature	5%	80%	13%	2%	Feature	11%	85%	2%	2%
Documentation	3%	10%	77%	10%	Documentation	–	6%	91%	3%
Question	28%	4%	4%	64%	Question	23%	2%	–	74%
	(2xA100 64 GB)					(4xA100 64 GB)			

Table 9
Confusion matrix for the two best-performing generative LLMs on the NLBSE'24 dataset. Rows represent actual classes, while columns correspond to predicted labels.

Meta-Llama-3-8B-Instruct	Bug	Feature	Question	*Mixtral-8x7B*	Bug	Feature	Question
Bug	93%	4%	3%	Bug	93%	2%	5%
Feature	15%	79%	6%	Feature	12%	71%	8%
Question	54%	13%	33%	Question	46%	10%	49%
	(2xA100 64 GB)				(4xA100 64 GB)		

we performed using the train set portion of the manually verified training set, with a training time of 4' 53'' for the best-performing baseline model (SETFIT based on issues). Conversely, Meta-LLama-3-8B-Instruct was queried with a zero-shot prompt. As for inference time, the SETFIT model requires 1'' to classify the entire test set, while the Meta-LLama-3-8B-Instruct takes 01 h 11' 04''.

For the 4xA100 64 GB group, Qwen2-72B-Instruct (F1 = .88) achieves the highest F1 compared to both the RoBERTa and SETFIT baselines. However, the only statistically significant difference is the one with RoBERTa, while the SETFIT performance is essentially equivalent to the one observed for Qwen2-72B-Instruct. As for the comparison between the two generative models, Qwen2-72B-Instruct is able to boost up the performance for the classes that are more difficult to classify, namely Documentation and Question, compared to the Meta-Llama-3-8B. However, albeit statistically significant, the improvement is negligible (+.01) compared to the best-performing baseline, i.e., the SETFIT model based on issues (F1 = .87).

NLBSE'24 test set. In Table 6, we report the performance obtained by the models on the NLBSE'24 test set. For the first group (2xA100 64 GB), the best-performing model is Meta-Llama-3-8B-Instruct (F1 macro = .66), having similar performance to gpt-4o-2024-05-13 (F1 macro = .63) on the same dataset. However, the response time for GPT-4o is lower (1 h 0' 41'') than the inference time observed for Meta-Llama (9 h 1' 25''). Qwen2-7B-Instruct has comparable performance (F1 macro = .62) but is able to better follow instructions (99% of parsable output) and has a better response time (1' 14' 54''), comparable to the one reported for GPT-4o (1' 00' 41''). For the second group of models (4xA100 64 GB), the best classification performance is achieved by Mixtral-8x7B-Instruct-v0.1 (F1 macro = .68), with a high percentage of parsable responses (93%). Qwen2-72B-Instruct and Meta-Llama-3-70B-Instruct have comparable performances (F1 macro = .67) but slightly better response times (3 h 59' 18'' and 4' 0' 49'', respectively). However, the performance of generative LLMs is substantially lower than the BERT-like baselines, with p-values indicating statistical differences. The comparison between the top-performing generative models for each group with RoBERTa and SETFIT is reported in Table 7,

Error analysis. To investigate the main causes of errors for both datasets, in the following we report the confusion matrices for the best-performing generative LLMs among those deployed on-premise. In particular, in Table 8, we report the confusion matrices for the two best-performing generative LLMs on the manually validated labels for the issues extracted from the NLBSE'23 dataset. Similarly, in Table 9, we report the confusion matrices for the two best-performing generative LLMs on the NLBSE'24 dataset.

In the first dataset, we observe as the main type of error the misclassification of *question* as *bug* for both models. The smaller LLM (i.e., Meta-Llama) also misclassifies *feature* as *documentation* in 13% of cases. *Documentation* is also confused with *feature* or *question* (10% of cases for both classes). These additional sources of confusion are less problematic for the second model, for which we observe lower percentages of misclassified issues. However, here, the misclassification of *feature* as *bug* is more frequent (from 5% with Meta-Llama to 11% with Qwen).

As for the second dataset, we report the confusion matrices in Table 9. It is interesting to note how the misclassification of *question* as *bug*, already observed for the first dataset, represents the most frequent type of error also for the second dataset. In particular, it is important to note that, despite the lower number of classes for this dataset, the proportion of misclassified cases is higher, up to double the number of cases for the Mixtral-8x7B on the NLBSE'24 dataset (46%) compared to what was observed for Qwen2-72B-Instruct for the first dataset (23%). Notably, we observe that the amount of misclassified questions is higher for both models than the correctly classified ones, with further *questions* erroneously labeled as *features* in 13% and 10% of cases, respectively. The misclassification of *feature* as *bug* is also observed for the second dataset for both models.

Table 10
Performance of LLMs on NLBSE'23 manually verified test set with the few-shot learning. The on-premise models are grouped by the hardware required. The best classification performance, response time, and percentage of parsable output are highlighted in bold.

One-shot learning									
2xA100 64 GB									
		F1							
Model	**Params**	**Bug**	**Docs**	**Feat**	**Ques**	**Micro**	**Macro**	**Time**	**% Parsable**
CodeLlama-7b-Instruct-hf	7b	0.22	0.31	**0.50**	**0.30**	**0.38**	**0.33**	00:53:03	0
starcoder2-7b	7b	0.38	**0.42**	0.03	0.07	0.28	0.23	01:01:22	0
Meta-Llama-3-8B-Instruct	8b	0.44	0.00	0.10	0.21	0.31	0.19	00:57:32	0
Mistral-7B-Instruct-v0.3	7b	**0.45**	0.00	0.10	0.12	0.32	0.17	00:33:32	0
zephyr-7b-beta	7b	0.44	0.00	0.12	0.04	0.29	0.15	01:12:43	0
CodeLlama-13b-Instruct-hf	13b	0.44	0.00	0.00	0.15	0.29	0.15	01:21:10	0
Qwen2-7B-Instruct	7b	0.41	0.06	0.10	0.00	0.27	0.14	00:12:36	0
deepseek-coder-6.7b-instruct	6.7b	0.44	0.00	0.00	0.04	0.28	0.12	01:08:10	0
gemma-7b-it	7b	0.43	0.00	0.00	0.04	0.28	0.12	**00:11:04**	0
Mistral-7B-Instruct-v0.2	7b	0.44	0.00	0.00	0.00	0.28	0.11	00:45:42	0
starcoder2-15b	15b	0.37	0.00	0.00	0.00	0.22	0.09	01:18:11	0
4xA100 64 GB									
Qwen2-72B-Instruct	72b	**0.84**	**0.81**	**0.82**	**0.72**	**0.80**	**0.80**	02:40:01	0
Mixtral-8 × 7B-Instruct-v0.1	46b	0.54	0.42	0.72	0.54	0.58	0.55	00:36:29	0
Codestral-22B-v0.1	22b	0.60	0.20	0.55	0.49	0.52	0.46	**00:31:52**	0
deepseek-coder-33b-instruct	33b	0.45	0.30	0.45	0.00	0.37	0.30	01:30:54	0
CodeLlama-34b-Instruct-hf	34b	0.40	0.00	0.04	0.21	0.26	0.16	01:21:14	0
Meta-Llama-3-70B-Instruct	70b	0.44	0.00	0.04	0.08	0.29	0.14	01:08:23	0
API Call									
gpt-4o-2024-05-13	–	0.74	0.71	0.85	0.59	0.74	0.72	00:08:07	100
Two-shot learning									
2xA100 64 GB									
		F1							
Model	**Params**	**Bug**	**Docs**	**Feat**	**Ques**	**Micro**	**Macro**	**Time**	**% Parsable**
CodeLlama-7b-Instruct-hf	7b	**0.48**	**0.22**	**0.45**	**0.19**	**0.40**	**0.34**	01:03:07	5
Meta-Llama-3-8B-Instruct	8b	0.44	0.00	0.10	0.04	0.29	0.15	00:43:46	17
CodeLlama-13b-Instruct-hf	13b	0.45	0.00	0.04	0.08	0.30	0.14	01:13:39	69
Qwen2-7B-Instruct	7b	0.44	0.12	0.00	0.00	0.29	0.14	00:11:04	11
deepseek-coder-6.7b-instruct	6.7b	0.43	0.06	0.04	0.00	0.28	0.13	01:06:02	0
gemma-7b-it	7b	0.43	0.00	0.00	0.08	0.28	0.13	**00:10:45**	12
Mistral-7B-Instruct-v0.3	7b	0.44	0.00	0.04	0.00	0.29	0.12	00:49:04	0
starcoder2-7b	7b	0.36	0.05	0.03	0.00	0.18	0.11	01:00:52	0
Mistral-7B-Instruct-v0.2	7b	0.44	0.00	0.00	0.00	0.28	0.11	00:55:45	0
zephyr-7b-beta	7b	0.44	0.00	0.00	0.00	0.28	0.11	01:09:02	0
starcoder2-15b	15b	0.35	0.00	0.04	0.00	0.22	0.10	01:17:42	0
4xA100 64 GB									
Qwen2-72B-Instruct	72b	**0.76**	**0.67**	**0.75**	**0.63**	**0.72**	**0.70**	02:16:23	92
Meta-Llama-3-70B-Instruct	70b	0.50	0.00	0.23	0.41	0.40	0.28	01:12:26	**7**
Mixtral-8 × 7B-Instruct-v0.1	46b	0.50	0.06	0.13	0.44	0.39	0.28	00:39:30	90
Codestral-22B-v0.1	22b	0.48	0.16	0.16	0.32	0.37	0.28	**00:34:23**	<1
deepseek-coder-33b-instruct	33b	0.46	0.18	0.28	0.08	0.31	0.25	01:35:20	12
CodeLlama-34b-Instruct-hf	34b	0.51	0.00	0.04	0.38	0.34	0.23	01:23:48	0
API Call									
gpt-4o-2024-05-13	–	0.82	0.88	0.87	0.77	0.83	0.83	00:09:10	100

4.2. Performance with few-shot learning

In the following, we report the classification performance with few-shot learning for both datasets in Table 10 and Table 11, respectively. We report only the models for which at least one instance was classified. The models that are omitted from the Tables are those that were not able to perform any label prediction for the issues in the test sets. This means that, despite employing both JSON parsing techniques and regular expression pattern matching, we were unable to successfully extract any of the target labels from the models' output. In Tables 12 and 13, we compare the top-performing generative models for each dataset when zero- and few-shot learning are used, and we report the classification performance by class.

Overall, we observe that using generative LLMs with a few-shot learning does not improve the classification performance, compared to the one observed in the zero-shot learning condition, with the only notable exception of GPT, for which a slight improvement is observed and for large models requiring 4GPU for deployment when evaluated on the NLBSE24 dataset. Conversely, a consistent drop in performance is observed when few-shot learning is used on the manually validated label dataset from NLBSE'23, compared to the zero-shot condition.

These findings suggests that few-shot learning may not necessarily be beneficial for automated issue labeling with generative models, in line with what we reported in our earlier study on the evaluation of GPT-like models for this task [14]. Regardless of the dataset being considered, the drop in F1-measure is more critical for smaller models

Table 11
Performance of LLMs on the NLBSE'24 test set with few-shot learning. The on-premise models are grouped by the hardware required. The best classification performance, response time, and percentage of parsable output are highlighted in bold.

One-shot learning								
2xA100 64 GB								
		F1						
Model	**Params**	**Bug**	**Feat**	**Que**	**Micro**	**Macro**	**Time**	**% Parsable**
deepseek-coder-6.7b-instruct	6.7b	**0.56**	**0.46**	**0.27**	**0.47**	**0.43**	08:48:02	0
Qwen2-7B-Instruct	7b	0.51	0.42	0.11	0.41	0.35	01:42:45	0
Meta-Llama-3-8B-Instruct	8b	0.49	0.15	0.31	0.38	0.31	04:20:58	<1
deepseek-coder-7b-instruct-v1.5	7b	0.25	0.40	0.27	0.31	0.31	06:45:38	0
Mistral-7B-Instruct-v0.3	7b	0.52	0.17	0.22	0.39	0.30	04:01:33	0
zephyr-7b-beta	7b	0.46	0.14	0.29	0.35	0.30	08:39:51	0
CodeLlama-7b-Instruct-hf	7b	0.52	0.11	0.25	0.39	0.30	08:16:17	0
gemma-7b-it	7b	0.52	0.18	0.18	0.38	0.29	**01:32:07**	0
starcoder2-7b	7b	0.41	0.13	0.21	0.29	0.25	08:07:37	0
CodeLlama-13b-Instruct-hf	13b	0.44	0.01	0.29	0.32	0.24	10:38:46	0
Mistral-7B-Instruct-v0.2	7b	0.50	0.00	0.04	0.34	0.18	04:45:28	0
starcoder2-15b	15b	0.45	0.00	0.01	0.29	0.15	10:20:15	0
4xA100 64 GB								
Qwen2-72B-Instruct	72b	**0.73**	**0.81**	0.58	**0.72**	**0.71**	04:57:02	0
Codestral-22B-v0.1	22b	0.60	0.65	**0.59**	0.61	0.61	**03:59:04**	0
Mixtral-8 × 7B-Instruct-v0.1	46b	0.58	0.66	0.46	0.57	0.57	06:20:57	0
Meta-Llama-3-70B-Instruct	70b	0.53	0.25	0.42	0.44	0.40	07:24:25	<1
CodeLlama-34b-Instruct-hf	34b	0.46	0.12	0.49	0.41	0.36	06:19:28	0
deepseek-coder-33b-instruct	33b	0.51	0.27	0.24	0.40	0.34	13:15:47	0
Llama-2-70b-chat-hf	70b	0.19	0.30	0.13	0.21	0.20	22:36:49	0
CodeLlama-70b-Instruct-hf	70b	0.02	0.08	0.02	0.04	0.04	26:12:02	**6**
API Call								
gpt-4o-2024-05-13	–	0.70	0.80	0.49	0.68	0.66	01:15:41	100
Two-shot learning								
2xA100 64 GB								
		F1						
Model	**Params**	**Bug**	**Feat**	**Que**	**Micro**	**Macro**	**Time**	**% Parsable**
deepseek-coder-6.7b-instruct	6.7b	**0.54**	**0.32**	**0.21**	**0.42**	**0.35**	08:16:37	<1
CodeLlama-13b-Instruct-hf	13b	0.50	0.01	0.37	0.38	0.29	08:43:00	24
Meta-Llama-3-8B-Instruct	8b	0.51	0.17	0.18	0.38	0.38	03:39:30	40
Mistral-7B-Instruct-v0.3	7b	0.52	0.17	0.12	0.38	0.27	04:05:31	0
CodeLlama-7b-Instruct-hf	7b	0.52	0.17	0.10	0.37	0.26	08:45:15	<1
zephyr-7b-beta	7b	0.50	0.08	0.15	0.36	0.24	07:29:49	0
gemma-7b-it	7b	0.50	0.08	0.10	0.35	0.23	01:32:37	83
starcoder2-7b	7b	0.49	0.03	0.09	0.31	0.21	08:09:09	0
Qwen2-7B-Instruct	7b	0.50	0.03	0.03	0.33	0.18	**01:24:09**	**98**
Mistral-7B-Instruct-v0.2	7b	0.50	0.00	0.00	0.33	0.17	07:51:49	0
starcoder2-15b	15b	0.42	0.04	0.02	0.26	0.16	10:38:52	0
4xA100 64 GB								
Qwen2-72B-Instruct	72b	**0.72**	**0.80**	**0.54**	**0.70**	**0.69**	18:53:48	18
Meta-Llama-3-70B-Instruct	70b	0.60	0.60	0.55	0.58	0.58	07:17:03	27
Codestral-22B-v0.1	22b	0.60	0.47	0.49	0.54	0.52	**04:22:39**	<1
deepseek-coder-33b-instruct	33b	0.53	0.30	0.16	0.41	0.33	14:30:30	1
CodeLlama-34b-Instruct-hf	34b	0.35	0.08	0.45	0.33	0.30	11:14:40	<1
Mixtral-8 × 7B-Instruct-v0.1	46b	0.51	0.02	0.15	0.36	0.23	06:34:55	87
API Call								
gpt-4o-2024-05-13	–	0.71	0.82	0.50	0.69	0.68	01:20:12	100

(see the comparison between the best-performing models in Tables 12 and 13.

4.3. Impact of window size on the prompts

In the following, we report a follow-up analysis aimed at investigating to what extent the token window size for each model might have an impact on the prompt construction, thus potentially affecting the classification performance. For each model, we calculate the average token length of the prompts, which includes the issue requiring classification. This analysis reveals that some models have input token size constraints that may be insufficient for longer prompts, resulting in prompt truncation and potentially compromising the reliability of results. To address these concerns, we calculated the percentage of issues in both test sets that undergo truncation due to token window size limitations. We examined both zero-shot and few-shot settings. The results are shown in Table 14. Note that each model has its own tokenizer, which might adopt slightly different tokenizing techniques, resulting in different average token lengths for the prompts, even in the presence of the same input texts. For this reason, for each model, we report a different average token length for the prompts. The results are presented in Table 14. The table shows that the majority of models

Table 12
Comparison of the performance of generative LLMs in the zero- and few-shot learning on the NLBSE'23 dataset.

	Zero Shot						One Shot						Two Shot					
	Meta-Llama-3-8B			Qwen2-72B			CodeLlama-7b			Qwen2-72B			CodeLlama-7b			Qwen2-72B		
Label	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
Bug	0.75	0.98	**0.85**	0.75	0.98	**0.85**	0.58	0.13	0.22	0.73	0.98	0.83	0.34	0.85	0.48	0.62	0.98	0.76
Documentation	0.88	0.64	0.74	0.95	0.74	**0.83**	0.30	0.31	0.31	0.89	0.75	0.81	0.36	0.16	0.22	0.89	0.53	0.67
Feature	0.90	0.80	0.85	0.94	0.85	**0.90**	0.38	0.75	0.5	0.86	0.78	0.82	0.85	0.31	0.45	0.78	0.73	0.75
Question	0.71	0.75	0.73	0.94	0.91	**0.92**	0.38	0.26	0.30	0.79	0.65	0.72	0.40	0.13	0.19	0.78	0.53	0.63
Micro avg	0.81	0.80	0.80	0.87	0.87	**0.87**	0.38	0.37	0.37	0.80	0.80	0.80	0.40	0.39	0.40	0.72	0.72	0.72
Macro avg	0.81	0.79	0.79	0.89	0.87	**0.88**	0.41	0.36	0.33	0.82	0.79	0.80	0.49	0.36	0.33	0.77	0.69	0.70
Hardware	2xA100 64 GB			4xA100 64 GB			2xA100 64 GB			4xA100 64 GB			2xA100 64 GB			4xA100 64 GB		
Inference Time	01:11:04			02:56:31			00:53:03			02:40:01			01:03:07			02:16:23		

Table 13
Comparison of the performance of generative LLMs in the zero- and few-shot learning on the NLBSE'24 dataset.

	Zero-shot						One-shot						Two-Shot					
	Meta-Llama-3-8B			Mixtral-8 × 7B			deepseek-coder-6.7b			Qwen2-72B			deepseek-coder-6.7b			Qwen2-72B		
Label	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
Bug	0.57	0.89	0.70	0.60	0.91	0.73	0.42	0.85	0.56	0.61	0.89	0.72	0.38	0.92	0.53	0.60	0.91	0.72
Feature	0.83	0.77	0.80	0.85	0.74	0.79	0.67	0.35	0.45	0.83	0.80	0.81	0.68	0.20	0.32	0.83	0.78	0.80
Question	0.72	0.36	0.48	0.70	0.41	0.52	0.46	0.19	0.27	0.79	0.45	0.58	0.53	0.13	0.21	0.77	0.41	0.54
Micro avg	0.68	0.67	0.68	0.69	0.69	0.69	0.47	0.46	0.47	0.72	0.71	0.72	0.42	0.42	0.42	0.70	0.70	0.70
Macro avg	0.71	0.67	0.68	0.72	0.69	0.68	0.51	0.46	0.43	0.74	0.71	0.71	0.53	0.42	0.35	0.73	0.70	0.69
Hardware	2xA100 64 GB			4xA100 64 GB			2xA100 64 GB			4xA100 64 GB			2xA100 64 GB			4xA100 64 GB		
Inference Time	09:01:25			04:58:03			08:48:02			04:57:02			09:04:42			18:53:48		

are rarely affected by truncation, especially in zero-shot learning. The prompt is more likely to be truncated for smaller models with 4096 token window size. This is particularly evident in the one- and two-shot settings, where the percentage of issues that undergo truncation is higher than in the zero-shot setting. In particular, we observe that truncation becomes prevalent only for models with a reduced context window size (4096 tokens), for which it reaches 100% of cases. However, for models with a higher window size (i.e., from 8192 on), the percentage of truncated prompts never exceeds 3% even in the few-shot learning condition.

Albeit marginal, the phenomenon of truncation appears associated with a significant drop in performance. In particular, the models for which we observe 100% of truncation cases report an F1 equal to zero,[4] with the only exception of the deepseek-coder-7b-instruct-v1.5 model, which achieves F1-macro = .31 in the one-shot setting on the NLBSE'24 dataset (see Table 11).

5. Discussion

Our results highlight the potential of leveraging generative LLMs for automated issue labeling, while also revealing some limitations. Our analysis goes beyond the assessment of classification accuracy through precision, recall, and F1 but also assesses response times, computational requirements, and the models' ability to follow the output formatting instructions, thus investigating the crucial trade-offs between human costs associated with manual labeling for training data and the computational costs of deploying large models. Also, we examine the relationship between model size and performance gains, as well as comparing zero- and few-shot versus supervised approaches. Based on the empirical evidence provided in this paper, in the following we answer our research question by also discussing the implications of our findings for research and practice.

[4] For the sake of space, we exclude these models from Tables 10 and 11. The full report with the classification performance for all models can be found in the replication package.

Generative vs. BERT-like LLMs. Our results complement and extend the evidence provided by our previous study [14], in which we provided a preliminary evaluation of the performance of GPT-like LLMs for automated issue labeling. In particular, in our former study, we observed that GPT-3.5 achieved performance comparable to the SETFIT baseline on the manually labeled dataset extracted from the NLBSE'23 dataset, which is one of the two datasets that we used also for this study. It is important to note that the SETFIT model was fine-tuned on a subset of the manually labeled gold standard dataset, while GPT-3.5 was used in a zero-shot setting and without fine-tuning. As such, we concluded our previous study by observing that GPT-like LLMs can be used to classify issue reports without any task-specific fine-tuning with a negligible loss in performance compared to BERT-like models, such as SETFIT, which is a significant advantage in the absence of labeled data.

In this study, we could only partially validate this previous finding. We replicated the evaluation using the currently available model (GPT-4o) and observed the same performance for the manually validated test set (F1 macro = .81 with zero-shot learning, as reported in Table 4). However, when evaluated on the second dataset (NLBSE'24), the classification performance of GPT drops substantially (F1 macro = .63), as reported in Table 6). This difference in performance for the two datasets is also observed for the generative LLMs that we deploy on-premise in the current benchmark, for which we observe a good classification performance on the manually verified set (F1 macro = .79 for Meta-Llama-3-8B-Instruct and F1 macro = .88 Qwen2-72B-Instruct, see Table 4). However, when evaluated on the NLBSE'24 dataset, all generative models report a substantial drop in performance (F1 macro = .63 for Meta-Llama-3-8B-Instruct and F1 macro = .68 Mixtral-8x7B-Instruct-v0.1, see Table 6). This difference in performance across datasets is also observed in the few-shot learning condition. SETFIT demonstrates good performance on both datasets, achieving F1 = 0.87 for the manually verified set and F1 = 0.82 on the NLBSE'24 (see Tables 5 and 7).

These findings suggest that, while generative LLMs used in a zero-shot (or few-shot) setting might appear as a viable solution in the absence (or with scarcity) of data for training, their performance can

Table 14
Token window size analysis across different models and their corresponding prompt truncation rates (%) for NLBSE'23 and NLBSE'24 datasets under zero-shot, one-shot, and two-shot settings.

			NLBSE'23					
			Zero Shot		One shot		Two Shot	
Hardware	Model	Context Window	Prompt Avg Tokens	% Trunc.	Prompt Avg Tokens	% Trunc.	Prompt Avg Tokens	% Trunc.
2xA100 64 GB	CodeLlama-13b-Instruct-hf	16384	858	–	5171	–	5947	–
	CodeLlama-7b-Instruct-hf	16384	858	–	5171	–	5947	–
	deepseek-coder-33b-instruct	16384	858	–	5109	–	5881	–
	deepseek-coder-6.7b-instruct	16384	858	–	5109	–	5881	–
	deepseek-coder-7b-instruct-v1.5	4096	818	2	4742	100	5481	100
	gemma-7b-it	8192	796	<1	4569	2	5269	2
	Llama-2-7b-chat-hf	4096	858	2	5171	100	5947	100
	Llama-2-13b-chat-hf	4096	858	2	5171	100	5947	100
	Meta-Llama-3-8B-Instruct	8192	703	–	3900	<1	4547	1
	Mistral-7B-Instruct-v0.2	32768	851	–	5159	–	5924	–
	Mistral-7B-Instruct-v0.3	32768	851	–	5159	–	5924	–
	Qwen2-7B-Instruct	32768	736	–	4083	–	4735	–
	starcoder2-7b	16384	802	–	4509	–	5214	–
	starcoder2-15b	16384	802	–	4509	–	5214	–
	zephyr-7b-beta	32768	851	–	5159	–	5924	–
4xA100 64 GB	CodeLlama-34b-Instruct-hf	16384	858	–	5171	–	5947	–
	CodeLlama-70b-Instruct-hf	16384	858	–	5171	–	5947	–
	Codestral-22B-v0.1	32768	851	–	5159	–	5924	–
	Llama-2-70b-chat-hf	4096	858	2	5171	100	5947	100
	Meta-Llama-3-70B-Instruct	8192	703	–	3900	<1	4547	1
	Mixtral-8 × 7B-Instruct-v0.1	32768	851	–	5159	–	5924	–
	Qwen2-72B-Instruct	32768	736	–	4083	–	4735	–
API call	gpt-4o-2024-05-13	128000	703	–	3928	–	4571	–
	gpt3.5-turbo-16k-0613	32768	702	–	3899	–	4546	–

			NLBSE'24					
			Zero Shot		One shot		Two Shot	
Hardware	Model	Context Window	Prompt Avg Tokens	% Trunc.	Prompt Avg Tokens	% Trunc.	Prompt Avg Tokens	% Trunc.
2xA100 64 GB	CodeLlama-13b-Instruct-hf	16384	1345	<1	5261	<1	6417	1
	CodeLlama-7b-Instruct-hf	16384	1345	<1	5261	<1	6417	1
	deepseek-coder-33b-instruct	16384	1352	<1	5268	1	6473	1
	deepseek-coder-6.7b-instruct	16384	1352	<1	5268	1	6473	1
	deepseek-coder-7b-instruct-v1.5	4096	1280	3	4889	97	6042	100
	gemma-7b-it	8192	1236	1	4707	2	5757	3
	Llama-2-7b-chat-hf	4096	1345	3	5261	100	6417	100
	Llama-2-13b-chat-hf	4096	1345	3	5261	100	6417	100
	Meta-Llama-3-8B-Instruct	8192	1062	1	3974	2	4797	2
	Mistral-7B-Instruct-v0.2	32768	1336	–	5162	<1	6304	<1
	Mistral-7B-Instruct-v0.3	32768	1336	–	5162	<1	6304	<1
	Qwen2-7B-Instruct	32768	1134	–	4376	–	5206	–
	starcoder2-7b	16384	1239	<1	4708	<1	5695	<1
	starcoder2-15b	16384	1239	<1	4708	<1	5695	<1
	zephyr-7b-beta	32768	1336	–	5162	<1	6304	<1
4xA100 64 GB	CodeLlama-34b-Instruct-hf	16384	1345	<1	5261	<1	6417	1
	CodeLlama-70b-Instruct-hf	16384	1345	<1	5261	<1	6417	1
	Codestral-22B-v0.1	32768	1336	–	5162	<1	6304	<1
	Llama-2-70b-chat-hf	4096	1345	3	5261	100	6417	100
	Meta-Llama-3-70B-Instruct	8192	1062	1	3974	2	4797	2
	Mixtral-8 × 7B-Instruct-v0.1	32768	1336	–	5162	<1	6304	<1
	Qwen2-72B-Instruct	32768	1134	–	4376	–	5206	–
API call	gpt-4o-2024-05-13	128000	1064	–	3963	–	4806	–
	gpt3.5-turbo-16k-0613	32768	1062	–	3974	–	4797	–

vary depending on the datasets. As for comparison between zero- and few-shot settings, we observe that when adding examples to the prompt some models tend to output nonsensical predictions, thus demonstrating their inability to address the classification task. This is probably due to the fact that the prompt becomes too long, and the model struggles to focus on the relevant information and follow the instructions [81]. This evidence aligns with findings from previous work discussing the fundamental challenge of few-shot learning, that is the inherent risk of model overfitting when constrained by limited and potentially unrepresentative training data. In scenarios with minimal examples, such as in-context learning, models frequently demonstrate a tendency to memorize the nuanced, context-specific patterns of the few available samples rather than extracting and learning robust, transferable conceptual representations, thus limiting the model's ability to generalize concepts when classifying new data [82].

For practical implementation, the finding of this empirical study suggests that zero-shot generative LLMs could serve as a viable solution in the absence of training data. However, their inconsistent performance across datasets suggests that a validation of their reliability is required before the deployment of classifiers based on generative LLMs. As such, a sanity check of the classification outcome is strongly recommended to ensure its consistency with research goals or with the

intended use of the classifier in practice. State-of-the-art performance for issue classification, however, can always be achieved by supervised training of classifiers based on smaller, encoder-only (i.e., BERT-like) LLMs.

The importance of data quality and internal consistency of labels. The drop in classification performance observed for the generative LLMs on the NLBSE'24 dataset, compared to the manually validated one, suggests that the effective classification performance depends on the internal consistency between train and test data. Despite being trained on billions of data points, generative LLMs might not always be able to achieve state-of-the-art performance in zero-shot settings. In contrast, fine-tuned BERT-like LLMs, such as SETFIT, demonstrate better classification performance by leveraging a consistent labeling rationale applied across both training and test data. The analysis of the confusion matrices reveals that misclassification of labels — particularly the erroneous categorization of *questions* as *bugs* — represents the most prevalent error type made by generative LLMs on the NLBSE'24 dataset, which contains raw labels as originally assigned in their respective projects. We can reasonably exclude poor data quality as a contributing factor affecting our results, given that the projects considered for the NLBSE'24 data collection are both popular and actively maintained. Rather, we conjecture that a cause for misclassification might reside in inconsistent labeling rationales across different projects, in line with evidence from previous research highlighting how threats to construct validity often constitute a significant factor affecting classification performance [11].

In this study, we include two benchmark datasets that are comparable in terms of data source, diversity, and collection criteria—both comprising issues collected from various GitHub projects for the NLBSE issue-labeling challenges. The main distinction between these datasets lies in their labeling methodology. The NLBSE'23 is obtained through manual verification of issue labels, with substantial inter-annotator agreement [7]. Conversely, the NLBSE'24 dataset incorporates issues with labels assigned directly by project developers, maintainers, and users. This direct assignment approach potentially introduces inconsistencies in label usage, as contributors might misclassify improvement requests as bugs and vice versa [83], or apply project-specific labeling guidelines [11]. Consequently, classifiers based on BERT-like LLMs such as RoBERTa and SETFIT — which require supervised training — tend to outperform zero-shot generative LLMs, which primarily leverage commonsense knowledge and depend solely on information provided in the prompt.

To label or not? The trade-off between computational costs vs. labeling costs. The cost analysis, in terms of computational resources, reveals that the ability of generative LLMs to achieve state-of-the-art performance achieved by BERT-like models, as observed for the manually validated data set, might not necessarily translate into a viable solution. In fact, we observe that the inference time might vary because of both model and dataset size, ranging from 9' for 7b models on the manually verified dataset to 25 h for 70b models on the NLBSE'24 dataset. This suggests that, in a real use case scenario, the choice of the model to adopt must be informed by a careful analysis of the cost associated with the model deployment, beyond the classification performance. Given the inconsistent cross-dataset performance of generative models for issue classification, organizations must carefully assess the potential reduction of costs associated with manual labeling of training data is favorable also considering the increased computational costs for their deployment on-premise.

We observe that the best-performing model might not necessarily be usable in practice due to the high inference time and the hardware requirements for deployment. Conversely, the inference time for SETFIT is significantly lower, ranging from 1'' to 9'' for the two datasets, respectively. This is even more important if we consider that SETFIT achieves comparable, state-of-the-art performance for both datasets, while a drop in F1 is observed for the generative LLMs on the NLBSE'24 dataset.

It is important to highlight that for BERT-like models, fine-tuning is a necessary step, albeit a relatively inexpensive process, both in terms of computational resources and human effort required for labeling the training data. Empirical evidence from this study shows how state-of-the-art performance can be achieved by using a training set of hundreds of labeled issues (see the performance of SETFIT in Table 5). In our study, this step required less than five minutes using the dataset of 200 manually labeled issues (see Table 5) and up to 40' for the larger NLBSE'24 that includes 1500 issues (see Table 7). In contrast, the cost associated with fine-tuning generative LLMs is significantly higher in terms of both computational resources and data required [84]. As such, the performance improvements gained from fine-tuning LLMs may not necessarily justify the associated computational costs.

Post-processing. Finally, another problem affecting generative LLMs is the need to post-process their output, as the output format does not always comply with the specifications provided in the prompt. This causes the need for additional computational effort required to parse the output to extract the label for issue classification. This was also investigated in previous work by Macedo and colleagues, who suggest prompt engineering and post-processing of responses required to have a better estimate of LLMs performance [49].

The aforementioned results suggest the importance of trading off the computational costs associated with the deployment of generative LLMs and post-processing of their output with the human costs associated with manually labeling training data for the fine-tuning of BERT-like LLMs.

6. Limitations

In the following, we discuss the limitations of this study. In doing so, we will explain how we addressed the trade-offs associated with the decision points in our study design [85], the threats of validity they might have introduced and how we mitigate them.

One potential threat to the *internal validity* of our study resides in the choice and design of the prompt template used for the generative LLMs. Indeed, previous work shows how prompt engineering is crucial to optimize the classification performance of GPT-like models [24]. In our previous study [14], we mitigate this threat by experimenting with multiple prompts of various lengths, i.e., by including the label definition only (zero-shot), or by appending examples of labeled issues (few-shot). Furthermore, we investigated various strategies for prompt definition in the few-prompt setting, i.e., by providing and omitting the description of the label along with the issue examples. In this study, we reuse the prompt strategy previously validated. Future work might follow up to further investigate effective prompting strategies with respect to the specific characteristics of the LLM being adopted.

Another threat to internal validity concerns internal factors, such as the configuration of the parameters, which is a known limitation of any ML-based approach. Specifically, when using generative LLMs, the model temperature has to be set, as high values in temperature might result in more unpredictable output (see, for example, the documentation of GPT-like models [86]). While this can be seen as an advantage in creative tasks (e.g., supporting dialogue-based interaction), it could be detrimental for classification tasks as the model can produce highly divergent outputs when the same prompt is provided [87]. In this study, we control for this factor by setting the temperature equal to zero for all the experimental conditions. This choice was made for several reasons: to ensure that the classification is consistent across different runs, which is crucial for the reproducibility of the results and gives a reliable estimate of the model performance. Second, having a deterministic output allows for a fair comparison between the models, as the output is not influenced by the randomness introduced by the temperature parameter. For this reason, a greedy sampling approach was preferred (meaning that top-p is set to 0 and top-k is set to 1). Future studies might consider investigating the impact of different values of temperature, top-p and top-k parameters for classification

tasks. Along the same line, further threats may have been introduced by the choice of considering only 4-bit quantization for the deployment of generative LLMs, which does make the LLMs easier to deploy on-premise but introduces the risk of observing a lower performance. While acknowledging this risk, our choice is in line with the overall goal of this study, that is to assess the performance of generative LLMs in a low-resource setting, with specific focus on the trade-off between the need for reducing the human costs associated with labeling of training data and the computational costs associated with the deployment of large generative models.

Another significant limitation concerns the evolving nature of commercial LLMs and their impact on research reproducibility. Specifically, our assessment of gpt-3.5-turbo was affected by model updates during the study period, which highlights a broader challenge in conducting research with proprietary language models. For instance, the model *gpt-3.5-turbo-16k-0613* was shut down by the provider, making it impossible to use it in the experiments for the NLBSE'24 dataset. Furthermore, the black-box nature of these models presents additional challenges for scientific inquiry. Unlike traditional machine learning approaches, where model architectures and training procedures are fully transparent, commercial LLMs often operate as closed systems with limited visibility into their internal workings, training data, or specific architectural choices. While we attempted to control for known parameters (such as temperature and sampling settings), the underlying model changes and the proprietary nature of these systems represent inherent limitations to the generalizability and replicability of our findings. To partially mitigate these concerns, we have documented the specific model version, enabling future researchers to at least contextualize our results within the appropriate model iterations. Additionally, our decision to focus on openly available alternatives alongside commercial models provides a more stable baseline for comparison, as these models' architectures and training procedures are typically more transparent and stable over time.

A further threat to internal validity may be introduced by the choice of using dataset with a 50:50 train-test split. This choice is justified by the need of enabling a fair comparison with previous studies that already implemented the same split. We acknowledge that in machine learning research, consolidated practice often favors different splitting strategies, such as 80:20 or 70:30 splits, which favor training data over test sets. However, when working with small datasets, as in the case of the NLBSE'23 dataset employed in our study, a 50:50 split can be preferred for the following reasons. First, it ensures that the test set is large enough to provide a statistically meaningful estimate of the model performance. With very small test sets, performance metrics can be highly variable and unreliable. Second, having an equal number of samples in both sets helps maintain similar statistical properties between them, potentially reducing sampling bias. Although cross-validation can be used to mitigate the risk of overfitting, the computational resources and inference time required for generative models poses significant computational challenges, thus limiting the application of a cross-validation approach.

Another potential limitation is introduced by the choice of datasets to include in our benchmark, which can limit the *conclusion validity* of this study. In this study, we decided to use datasets from the issue-labeling challenges organized in two editions of NLBSE workshops. We acknowledge that our methodology could produce different results if applied to different datasets as our findings may not necessarily generalize to data from other platforms, different from Github. In this perspective, the choice of focusing on Github issues might have limited the *external validity* of our findings, thus calling for further replications to confirm our conclusions.

Finally, as a further limitation, we acknowledge that a better classification performance could have been achieved by fine-tuning the generative LLMs using data from the software development domain. Albeit interesting to investigate, we consider this out of scope for the current benchmark, in which we are interested in investigating the trade-off between costs associated with manual labeling of limited training data required to implement BERT-based classifiers and the computational costs associated with the deployment of generative LLMs that can be used in a zero- or few-shot setting, i.e. in the absence or scarcity of training data. Future replications involving the fine-tuning of generative models could complement the evidence provided in this paper, thus providing complete and more comprehensive empirical evidence to inform researchers' and practitioners' decisions.

7. Conclusion

In this study, we have investigated to what extent LLMs can be leveraged for automatic issue labeling in a low-resource setting where it is crucial to address the trade-off between the human costs associated with labeling high-quality training data and the computational costs associated with the deployment of generative LLMs that can be used in a zero-shot setting, i.e. in absence of training data. Specifically, we have investigated the classification performance, response time, and quality of the responses for 22 generative LLMs of different sizes (from $\sim$ 7 to $\sim$ 70 billion parameters), which we compare with state-of-the-art BERT-like encoder-only models, using two datasets of issues collected from GitHub. For the sake of completeness, we also evaluated the performance of generative LLMs from the GPT family.

Empirical evidence provided by this study uncovers significant trade-offs in model selection. We observe how generative LLMs used in a zero-shot setting could represent a viable solution for automated issue labeling in the absence of resources for manually labeling a gold standard dataset. However, a sanity check is recommended as the generative LLMs showed inconsistent behavior across the two datasets. In terms of performance, zero-shot usage of generative LLMs demonstrated notable but inconsistent results. While they are able to achieve state-of-the-art accuracy on a manually verified dataset (with an F1 score reaching 0.88), their performance declines significantly when tested on a different dataset with crowd-sourced labels (with F1 score achieving 0.68 by the best-performing model). This variability raises important questions about their reliability across different use cases and projects.

Further limitations to the adoption of generative LLMs for automated issue labeling are represented by the computational costs for their deployment and the inference time, as large generative models may required substantial computational resources and exhibited significantly longer inference times compared to encoder-only models, sometimes extending to 25 h for certain datasets. These resource demands must be weighed against the potential benefits in terms of classification performance. We found that BERT-like models offered a more reliable alternative, demonstrating consistent classification performance across different datasets. These models achieved comparable or superior results while demanding significantly fewer computational resources. Notably, they could achieve good performance with relatively small amounts of labeled training data, requiring only hundreds of examples for effective fine-tuning. Furthermore, BERT-like models exhibit considerably lower inference time, which makes them eligible for adoption in use cases where the model response time is critical.

To further evaluate the generalizability of our findings, we plan to perform future replications with larger and more varied benchmark datasets, including issues from different projects and platforms, also with additional experimental conditions, including the fine-tuning of generative LLMs and prompt engineering. Furthermore, we acknowledge the need for further investigation to improve the consistency of generative LLMs across varying datasets. Research in this field could benefit from exploring hybrid approaches that leverage the strengths of both generative and BERT-like models. Additionally, further study could explore approaches to reducing the computational overhead of adopting generative LLMs to improve the feasibility of their on-premise deployment for widespread adoption in software development.

While generative LLMs emerge as promising for automated issue labeling, their practical implementation requires careful consideration of human and computational resource constraints that have to be weighted against classification performance requirements. For many organizations, traditional BERT-like models may represent the most practical solution, offering consistent performance and manageable resource requirements.

CRediT authorship contribution statement

Giuseppe Colavito: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Data curation, Conceptualization. **Filippo Lanubile:** Writing – review & editing, Writing – original draft, Supervision, Software, Resources, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Nicole Novielli:** Writing – review & editing, Writing – original draft, Supervision, Software, Resources, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was co-funded by the NRRP Initiative, Mission 4, Component 2, Investment 1.3 - Partnerships extended to universities, research centres, companies and research D.D. MUR n. 341 del 15.03.2022 – Next Generation EU (“FAIR - Future Artificial Intelligence Research”, code PE00000013, CUP H97G22000210007) and by the European Union - NextGenerationEU through the Italian Ministry of University and Research, Projects PRIN 2022 (“QualAI: Continuous Quality Improvement of AI-based Systems”, grant n. 2022B3BP5S, CUP: H53D23003510006). The models are deployed on the Leonardo supercomputer with the support of CINECA-Italian Super Computing Resource Allocation, class C project LLM4SE (HP10C7ORC9).

Data availability

The replication material is available at: https://github.com/collab-uniba/PromptingLLMs All datasets are publicly available and distributed in the scope of previous work, as mentioned in the paper.

References

[1] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, Y.-G. Guéhéneuc, Is it a bug or an enhancement? a text-based approach to classify change requests, in: Proceedings of the 2008 Conf. of the Center for Advanced Studies on Collaborative Research: Meeting of Minds, ACM, New York, NY, USA, 2008, http://dx.doi.org/10.1145/1463788.1463819.

[2] N. Pandey, D. Sanyal, A. Hudait, A. Sen, Automated classification of software issue reports using machine learning techniques: an empirical study, Innov. Syst. Softw. Eng. (2017) http://dx.doi.org/10.1007/s11334-017-0294-1.

[3] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: J. Burstein, C. Doran, T. Solorio (Eds.), Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Association for Computational Linguistics, Minneapolis, Minnesota, 2019, pp. 4171–4186, http://dx.doi.org/10.18653/v1/N19-1423.

[4] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, R. Soricut, ALBERT: A lite BERT for self-supervised learning of language representations, 2020, arXiv:1909.11942.

[5] V. Sanh, L. Debut, J. Chaumond, T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2020, arXiv:1910.01108.

[6] L. Zhuang, L. Wayne, S. Ya, Z. Jun, A robustly optimized BERT pre-training approach with post-training, in: S. Li, M. Sun, Y. Liu, H. Wu, K. Liu, W. Che, S. He, G. Rao (Eds.), Proceedings of the 20th Chinese National Conference on Computational Linguistics, Chinese Information Processing Society of China, Huhhot, China, 2021, pp. 1218–1227, URL: https://aclanthology.org/2021.ccl-1.108.

[7] G. Colavito, F. Lanubile, N. Novielli, Few-shot learning for issue report classification, in: 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering, NLBSE, 2023, pp. 16–19, http://dx.doi.org/10.1109/NLBSE59153.2023.00011.

[8] G. Colavito, F. Lanubile, N. Novielli, Issue report classification using pre-trained language models, in: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering, NLBSE ’22, Association for Computing Machinery, New York, NY, USA, 2023, pp. 29–32, http://dx.doi.org/10.1145/3528588.3528659.

[9] M. Izadi, CatIss: An intelligent tool for categorizing issues reports using transformers, in: NLBSE 2022, 2022, http://dx.doi.org/10.1145/3528588.3528662.

[10] M. Izadi, K. Akbari, A. Heydarnoori, Predicting the objective and priority of issue reports in software repositories, Empir. Softw. Eng. (2) (2022) http://dx.doi.org/10.1007/s10664-021-10085-3.

[11] G. Colavito, F. Lanubile, N. Novielli, L. Quaranta, Impact of data quality for automatic issue classification using pre-trained language models, J. Syst. Softw. 210 (2024) 111838, http://dx.doi.org/10.1016/j.jss.2023.111838, URL: https://www.sciencedirect.com/science/article/pii/S0164121223002339.

[12] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J.M. Zhang, Large language models for software engineering: Survey and open problems, in: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering, ICSE-FoSE, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 31–53, http://dx.doi.org/10.1109/ICSE-FoSE59343.2023.00008.

[13] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, 2023, arXiv:2308.10620.

[14] G. Colavito, F. Lanubile, L. Quaranta, N. Novielli, Leveraging GPT-like LLMs to automate issue labeling, in: Proceedings of 21st International Conference on Mining Software Repositories, MSR 2024, April 2024, 2024, http://dx.doi.org/10.1145/3643991.3644903.

[15] OpenAI, ChatGPT: Optimizing Language Models for Dialogue, OpenAI, 2022, URL: https://platform.openai.com/docs/models#gpt-3-5-turbo.

[16] R. Kallis, M. Izadi, L. Pascarella, O. Chaparro, P. Rani, The NLBSE’23 tool competition, in: 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering, NLBSE, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 1–8, http://dx.doi.org/10.1109/NLBSE59153.2023.00007.

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L.u. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017, URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[18] OpenAI, GPT-4 technical report, 2023, arXiv:2303.08774.

[19] W.X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, J.-R. Wen, A survey of large language models, 2023, arXiv:2303.18223.

[20] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, Nature 323 (6088) (1986) 533–536, http://dx.doi.org/10.1038/323533a0.

[21] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, http://dx.doi.org/10.1162/neco.1997.9.8.1735.

[22] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving Language Understanding by Generative Pre-Training, OpenAI, 2018, URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

[23] P. He, X. Liu, J. Gao, W. Chen, DeBERTa: Decoding-enhanced BERT with disentangled attention, 2021, arXiv:2006.03654.

[24] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), in: Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 1877–1901, URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[25] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and efficient foundation language models, 2023, arXiv:2302.13971.

[26] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C.C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P.S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E.M. Smith, R. Subramanian, X.E. Tan, B. Tang, R. Taylor, A. Williams, J.X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, T. Scialom, Llama 2: Open foundation and fine-tuned chat models, 2023, arXiv:2307.09288.

[27] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H.W. Chung, C. Sutton, S. Gehrmann, et al., PaLM: Scaling language modeling with pathways, J. Mach. Learn. Res. 24 (240) (2023) 1–113, URL: http://jmlr.org/papers/v24/22-1144.html.

[28] G. Team, Gemini: A family of highly capable multimodal models, 2024, arXiv:2312.11805, URL: https://arxiv.org/abs/2312.11805.

[29] W.L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J.E. Gonzalez, I. Stoica, E.P. Xing, Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality, 2023.

[30] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, É. Goffinet, D. Hesslow, J. Launay, Q. Malartic, D. Mazzotta, B. Noune, B. Pannier, G. Penedo, The falcon series of open language models, 2023, arXiv:2311.16867, URL: https://arxiv.org/abs/2311.16867.

[31] A.Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D.S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L.R. Lavaud, M.-A. Lachaux, P. Stock, T.L. Scao, T. Lavril, T. Wang, T. Lacroix, W.E. Sayed, Mistral 7B, 2023, arXiv:2310.06825.

[32] L. Tunstall, E. Beeching, N. Lambert, N. Rajani, K. Rasul, Y. Belkada, S. Huang, L. von Werra, C. Fourrier, N. Habib, N. Sarrazin, O. Sanseviero, A.M. Rush, T. Wolf, Zephyr: Direct distillation of LM alignment, 2023, arXiv:2310.16944.

[33] J. Kaplan, S. McCandlish, T. Henighan, T.B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models, 2020, arXiv:2001.08361.

[34] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E.H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, W. Fedus, Emergent abilities of large language models, 2022, arXiv:2206.07682, URL: https://jmlr.org/papers/volume21/20-074/20-074.pdf.

[35] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, 2020, pp. 1536–1547, http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139, Online.

[36] J. Tabassum, M. Maddela, W. Xu, A. Ritter, Code and named entity recognition in StackOverflow, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL, 2020, http://dx.doi.org/10.18653/v1/2020.acl-main.443, URL: https://aclanthology.org/2020.acl-main.443.

[37] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P.J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, J. Mach. Learn. Res. 21 (1) (2020) URL: https://jmlr.org/papers/volume21/20-074/20-074.pdf.

[38] Y. Wang, W. Wang, S. Joty, S.C. Hoi, CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 8696–8708, http://dx.doi.org/10.18653/v1/2021.emnlp-main.685.

[39] B. Yetistiren, I. Ozsoy, E. Tuzun, Assessing the quality of GitHub copilot's code generation, in: Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering, in: PROMISE 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 62–71, http://dx.doi.org/10.1145/3558489.3559072.

[40] H. Yu, B. Shen, D. Ran, J. Zhang, Q. Zhang, Y. Ma, G. Liang, Y. Li, Q. Wang, T. Xie, CoderEval: A benchmark of pragmatic code generation with generative pre-trained models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024, http://dx.doi.org/10.1145/3597503.3623316.

[41] GitHub, GitHub copilot now has a better AI model and new capabilities, 2023, URL: https://github.blog/ai-and-ml/github-copilot/github-copilot-now-has-a-better-ai-model-and-new-capabilities/.

[42] J. Cao, M. Li, M. Wen, S. chi Cheung, A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair, 2023, arXiv:2304.08191.

[43] Y. Charalambous, N. Tihanyi, R. Jain, Y. Sun, M.A. Ferrag, L.C. Cordeiro, A new era in software security: Towards self-healing software via large language models and formal verification, 2023, arXiv:2305.14752.

[44] S.B. Hossain, N. Jiang, Q. Zhou, X. Li, W.-H. Chiang, Y. Lyu, H. Nguyen, O. Tripp, A deep dive into large language models for automated bug localization and repair, Proc. ACM Softw. Eng. 1 (FSE) (2024) http://dx.doi.org/10.1145/3660773.

[45] M. Lajkó, V. Csuvik, L. Vidács, Towards JavaScript program repair with generative pre-trained transformer (GPT-2), in: 2022 IEEE/ACM International Workshop on Automated Program Repair, APR, 2022, pp. 61–68, http://dx.doi.org/10.1145/3524459.3527350.

[46] D. Sobania, M. Briesch, C. Hanna, J. Petke, An analysis of the automatic bug fixing performance of ChatGPT, in: 2023 IEEE/ACM International Workshop on Automated Program Repair, APR, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 23–30, http://dx.doi.org/10.1109/APR59189.2023.00012.

[47] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y.T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M.T. Ribeiro, Y. Zhang, Sparks of artificial general intelligence: Early experiments with GPT-4, 2023, arXiv:2303.12712.

[48] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P.d. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F.P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W.H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A.N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code, 2021, arXiv:2107.03374.

[49] M. Macedo, Y. Tian, F. Cogo, B. Adams, Exploring the impact of the output format on the evaluation of large language models for code translation, in: Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering, FORGE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 57–68, http://dx.doi.org/10.1145/3650105.3652301.

[50] Z. Yang, F. Liu, Z. Yu, J.W. Keung, J. Li, S. Liu, Y. Hong, X. Ma, Z. Jin, G. Li, Exploring and unleashing the power of large language models in automated code translation, Proc. ACM Softw. Eng. 1 (FSE) (2024) http://dx.doi.org/10.1145/3660778.

[51] J. Li, Y. Zhang, Z. Karas, C. McMillan, K. Leach, Y. Huang, Do machines and humans focus on similar code? Exploring explainability of large language models in code summarization, in: 2024 IEEE/ACM 32nd International Conference on Program Comprehension, ICPC, IEEE Computer Society, Los Alamitos, CA, USA, 2024, pp. 47–51, URL: https://doi.ieeecomputersociety.org/.

[52] J. Zhang, X. Wang, H. Zhang, H. Sun, X. Liu, Retrieval-based neural source code summarization, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1385–1397, http://dx.doi.org/10.1145/3377811.3380383.

[53] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang, Software testing with large language models: Survey, landscape, and vision, IEEE Trans. Softw. Eng. 50 (4) (2024) 911–936, http://dx.doi.org/10.1109/TSE.2024.3368208.

[54] S. Dou, J. Shan, H. Jia, W. Deng, Z. Xi, W. He, Y. Wu, T. Gui, Y. Liu, X. Huang, Towards understanding the capability of large language models on code clone detection: A survey, 2023, arXiv:2308.01191.

[55] Z. Yuan, J. Liu, Q. Zi, M. Liu, X. Peng, Y. Lou, Evaluating instruction-tuned large language models on code comprehension and generation, 2023, arXiv:2308.01240.

[56] T.F. Bissyandé, D. Lo, L. Jiang, L. Réveillère, J. Klein, Y.L. Traon, Got issues? Who cares about it? A large scale investigation of issue trackers from GitHub, in: 2013 IEEE 24th Int'L Symposium on Software Reliability Engineering, ISSRE, 2013, http://dx.doi.org/10.1109/ISSRE.2013.6698918.

[57] Q. Fan, Y. Yu, G. Yin, T. Wang, H. Wang, Where is the road for issue reports classification based on text mining? in: 2017 ACM/IEEE Int'L Symposium on Empirical Software Engineering and Measurement, ESEM, 2017, http://dx.doi.org/10.1109/ESEM.2017.19.

[58] S. Panichella, G. Bavota, M.D. Penta, G. Canfora, G. Antoniol, How developers' collaborations identified from different sources tell us about code changes, in: 2014 IEEE International Conference on Software Maintenance and Evolution, 2014, http://dx.doi.org/10.1109/ICSME.2014.47.

[59] J.L. Cánovas Izquierdo, V. Cosentino, B. Rolandi, A. Bergel, J. Cabot, GiLA: GitHub label analyzer, in: 2015 IEEE 22nd Int'L Conf.on Software Analysis, Evolution, and Reengineering, SANER, 2015, http://dx.doi.org/10.1109/SANER.2015.7081860.

[60] Z. Liao, D. He, Z. Chen, X. Fan, Y. Zhang, S. Liu, Exploring the characteristics of issue-related behaviors in GitHub using visualization techniques, IEEE Access (2018) http://dx.doi.org/10.1109/ACCESS.2018.2810295.

[61] K. Herzig, S. Just, A. Zeller, It's not a bug, it's a feature: How misclassification impacts bug prediction, in: 2013 35th International Conference on Software Engineering, ICSE, 2013, pp. 392–401, http://dx.doi.org/10.1109/ICSE.2013.6606585.

[62] R. Kallis, A. Di Sorbo, G. Canfora, S. Panichella, Ticket tagger: Machine learning driven issue classification, in: 2019 IEEE Intl. Conf. on Software Maintenance and Evolution,, ICSME 2019, Cleveland, OH, USA, September 29 - October 4, 2019, IEEE, 2019, http://dx.doi.org/10.1109/ICSME.2019.00070.

[63] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching word vectors with subword information, Trans. Assoc. Comput. Linguist. 5 (2017) 135–146, http://dx.doi.org/10.1162/tacl_a_00051.

[64] R. Kallis, A. Di Sorbo, G. Canfora, S. Panichella, Predicting issue types on GitHub, Sci. Comput. Program. (2021) http://dx.doi.org/10.1016/j.scico.2020.102598.
[65] Y. Zhou, Y. Tong, R. Gu, H. Gall, Combining text mining and data mining for bug report classification, J. Softw. Evol. Process. 28 (3) (2016) 150–176, http://dx.doi.org/10.1002/smr.1770.
[66] W. Alhindi, A. Aleid, I. Jenhani, M. Mkaouer, Issue-labeler: an ALBERT-based jira plugin for issue classification, in: 2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems, MOBILESoft, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 40–43, http://dx.doi.org/10.1109/MOBILSoft59058.2023.00012.
[67] J. Wang, X. Zhang, L. Chen, How well do pre-trained contextual language representations recommend labels for GitHub issues? Knowl.-Based Syst. (2021) http://dx.doi.org/10.1016/j.knosys.2021.107476.
[68] G. Aracena, K. Luster, F. Santos, I. Steinmacher, M.A. Gerosa, Applying large language models to issue classification, in: Proceedings of the Third ACM/IEEE International Workshop on NL-Based Software Engineering, NLBSE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 57–60, http://dx.doi.org/10.1145/3643787.3648043.
[69] A.J. Viera, J.M. Garrett, Understanding interobserver agreement: the kappa statistic, Fam. Med. (2005) URL: https://pubmed.ncbi.nlm.nih.gov/15883903/.
[70] R. Kallis, M. Izadi, L. Pascarella, O. Chaparro, P. Rani, The NLBSE'23 tool competition, in: 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering, NLBSE, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 1–8, http://dx.doi.org/10.1109/NLBSE59153.2023.00007.
[71] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, A. Rush, Transformers: State-of-the-art natural language processing, in: Q. Liu, D. Schlangen (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, 2020, pp. 38–45, http://dx.doi.org/10.18653/v1/2020.emnlp-demos.6, Online. URL: https://aclanthology.org/2020.emnlp-demos.6.
[72] Tim Dettmers, bitsandbytes, 2021, URL: https://github.com/TimDettmers/bitsandbytes (Accessed 17 June 2024).
[73] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q.V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), in: Advances in Neural Information Processing Systems, vol. 35, Curran Associates, Inc., 2022, pp. 24824–24837, URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.
[74] T. Zhang, I.C. Irsan, F. Thung, D. Lo, Revisiting sentiment analysis for software engineering in the era of large language models, ACM Trans. Softw. Eng. Methodol. (2024) http://dx.doi.org/10.1145/3697009.
[75] F. Sebastiani, Machine learning in automated text categorization, ACM Comput. Surv. 34 (1) (2002) 1–47, http://dx.doi.org/10.1145/505282.505283.
[76] L. Tunstall, N. Reimers, U.E.S. Jo, L. Bates, D. Korat, M. Wasserblat, O. Pereg, Efficient few-shot learning without prompts, 2022, http://dx.doi.org/10.48550/arXiv.2209.11055.
[77] T.G. Dietterich, Approximate statistical tests for comparing supervised classification learning algorithms, Neural Comput. 10 (7) (1998) 1895–1923, http://dx.doi.org/10.1162/089976698300017197.
[78] K.A. Alam, A. Jumani, H. Aamir, M. Uzair, ClassifAI: Automating issue reports classification using pre-trained BERT (Bidirectional Encoder Representations from Transformers) models, in: Proceedings of the Third ACM/IEEE International Workshop on NL-Based Software Engineering, NLBSE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 49–52, http://dx.doi.org/10.1145/3643787.3648041.
[79] R. Kallis, O. Chaparro, A. Di Sorbo, S. Panichella, NLBSE'22 tool competition, in: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering, NLBSE '22, Association for Computing Machinery, New York, NY, USA, 2023, pp. 25–28, http://dx.doi.org/10.1145/3528588.3528664.
[80] K. Song, X. Tan, T. Qin, J. Lu, T.-Y. Liu, MPNet: Masked and permuted pre-training for language understanding, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), in: Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 16857–16867, URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/c3a690be93aa602ee2dc0ccab5b7b67e-Paper.pdf.
[81] N.F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Trans. Assoc. Comput. Linguist. 12 (2024) 157–173, http://dx.doi.org/10.1162/tacl_a_00638, URL: https://aclanthology.org/2024.tacl-1.9.
[82] J. Zhang, H. Gao, P. Zhang, B. Feng, W. Deng, Y. Hou, LA-UCL: LLM-augmented unsupervised contrastive learning framework for few-shot text classification, in: N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, N. Xue (Eds.), Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, LREC-COLING 2024, ELRA and ICCL, Torino, Italia, 2024, pp. 10198–10207, URL: https://aclanthology.org/2024.lrec-main.890/.
[83] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, Y.G. Guéhéneuc, Is it a bug or an enhancement? A text-based approach to classify change requests, in: Proc. of the 2008 Conf. of the Center for Advanced Studies on Collaborative Research: Meeting of Minds, CASCON '08, ACM, New York, NY, USA, 2008, http://dx.doi.org/10.1145/1463788.1463819.
[84] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient finetuning of quantized LLMs, 2023, arXiv:2305.14314, URL: https://arxiv.org/abs/2305.14314.
[85] M.P. Robillard, D.M. Arya, N.A. Ernst, J.L.C. Guo, M. Lamothe, M. Nassif, N. Novielli, A. Serebrenik, I. Steinmacher, K.-J. Stol, Communicating study design trade-offs in software engineering, ACM Trans. Softw. Eng. Methodol. 33 (5) (2024) http://dx.doi.org/10.1145/3649598.
[86] OpenAI, API documentation, how should I set the temperature parameter?, 2023, https://platform.openai.com/docs/guides/text-generation/how-should-i-set-the-temperature-parameter (Accessed 17 November 2023).
[87] F.F. Xu, U. Alon, G. Neubig, V.J. Hellendoorn, A systematic evaluation of large language models of code, in: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, in: MAPS 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1–10, http://dx.doi.org/10.1145/3520312.3534862.